

# MATHEMATICAL FOUNDATIONS

Computational Imaging – Spring 2024

Ana Serrano, Albert Redó, Julio Marco

Master in Robotics, Graphics and Computer Vision – Universidad de Zaragoza

# Slide credits

Many of these slides are based on the following content:

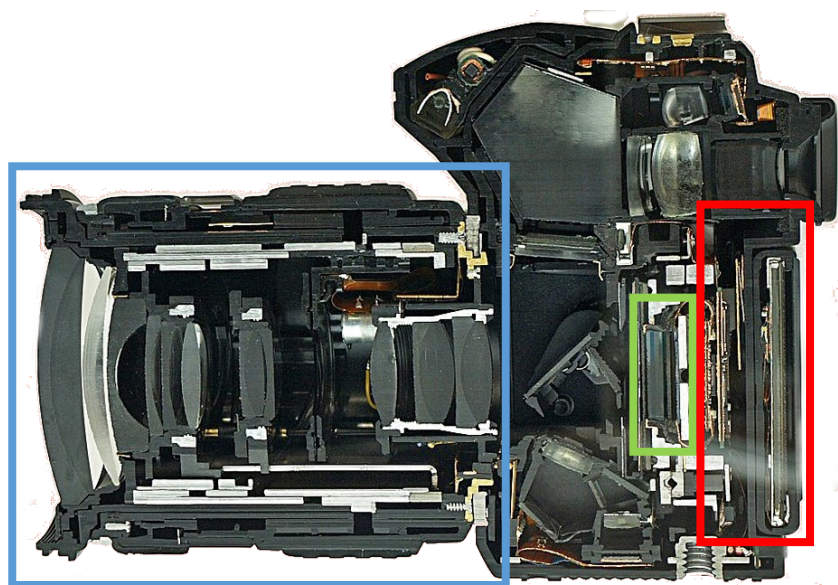
- Fredo Durand @MIT
- Gordon Wetzstein @Stanford University
- Ioannis Gkioulekas @CMU
- Wojciech Jarosz @Dartmouth University
- James Topmkin @Brown University

# Previously on **COMPUTATIONAL IMAGING**

# The modern photography pipeline



post-capture processing



optics and  
optical controls



sensor, analog  
front-end, and  
color filter array



in-camera image  
processing  
pipeline

# This week ...

- Defocus: Why it is important, why it is challenging
  - **Convolution**
- Frequency domain and the Fourier Transform
  - **Fourier domain**: convolution & deconvolution
- Alternative domains and sparsity
  - Gradient domain

# Why are our images blurry?

- Lens imperfections (aberrations, diffraction...)
- Camera shake (motion blur)
- Scene motion (more motion blur)
- Depth defocus (out-of-focus areas)

BLUR, SHARPEN



Scene motion

# Depth defocus



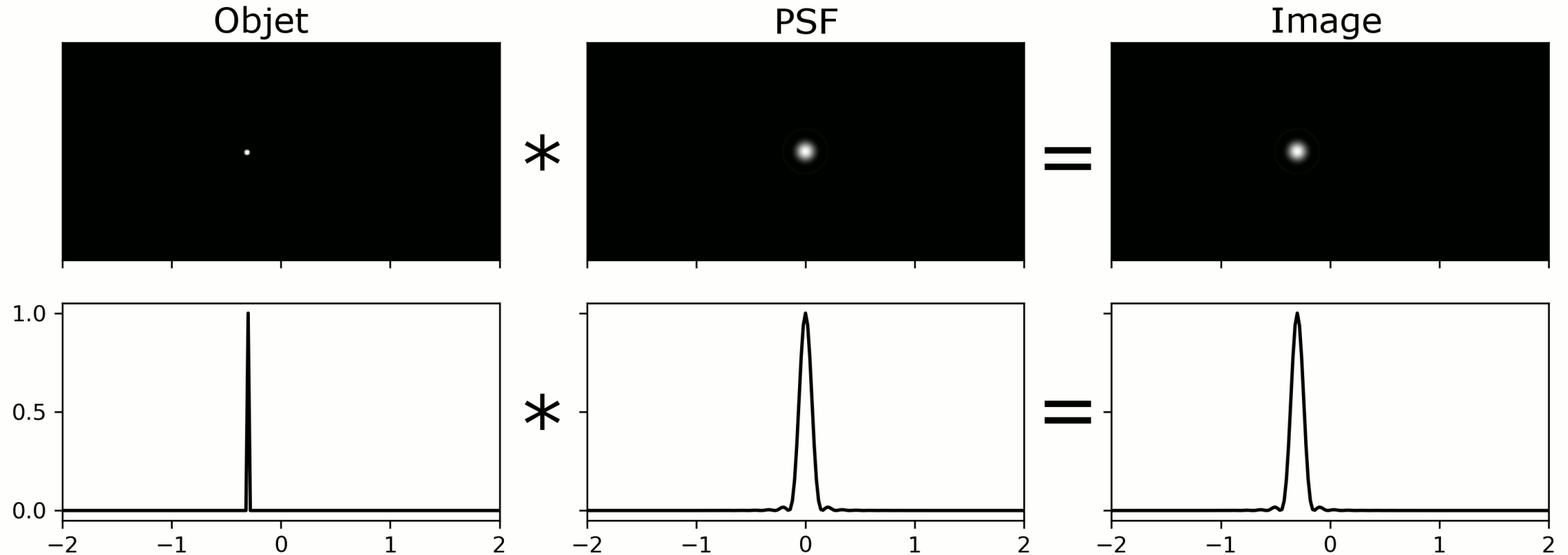
# Why are our images blurry?

- Lens imperfections (aberrations, diffraction...)
- Camera shake (motion blur)
- Scene motion (more motion blur)
- Depth defocus (out-of-focus areas)

BLUR, SHARPEN

# The point spread function (PSF)

Point spread function (PSF): The blur kernel of a lens.



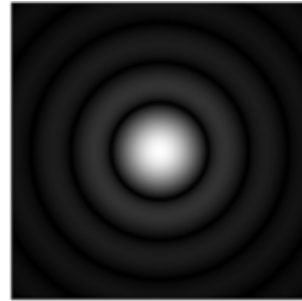
# Lens as an optical low-pass filter



image from a perfect lens

I

\*



=



image from imperfect lens

\*

k

=

B

# Image Processing Motivation

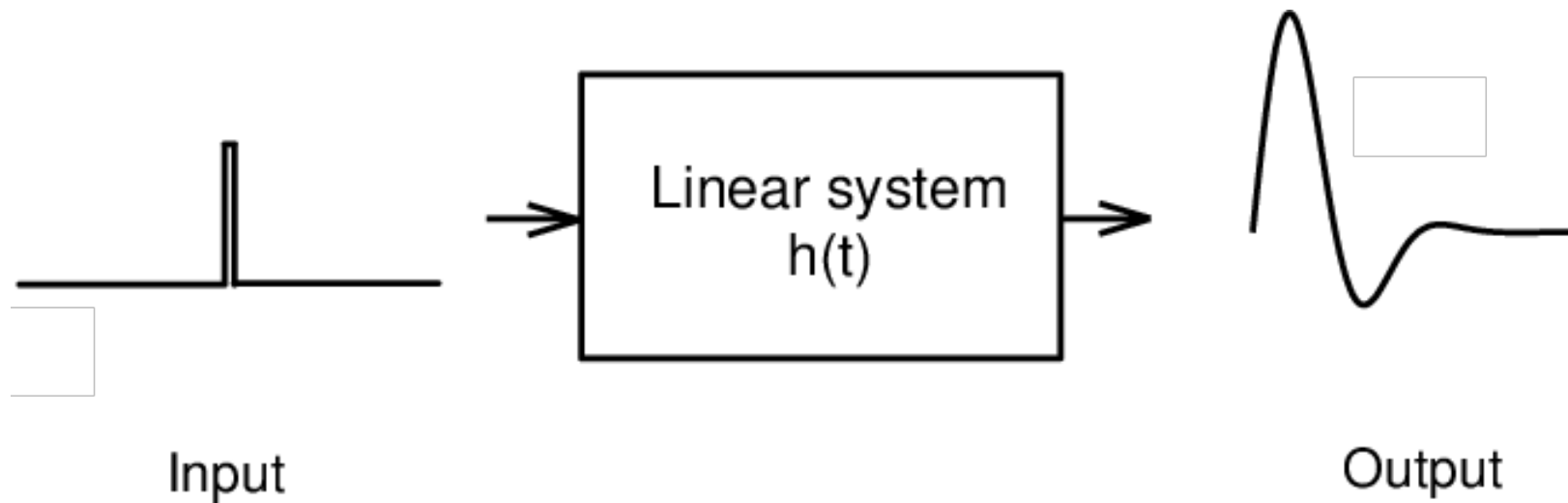
- We **must** understand convolution
- Can we remove it computationally?
  - Inverse the blur equation → **Deconvolution**

# CONVOLUTION 101

# Convolution: formal definition

Convolution – Allows us to “combine” two functions expressing **how the shape of one is modified by the other**

- Used to characterize linear time-invariant (LTI) systems (impulse response)



# Convolution: formal definition

Convolution – Allows us to “combine” two functions expressing how the shape of one is modified by the other

- Used to characterize linear time-invariant (LTI) systems (impulse response)

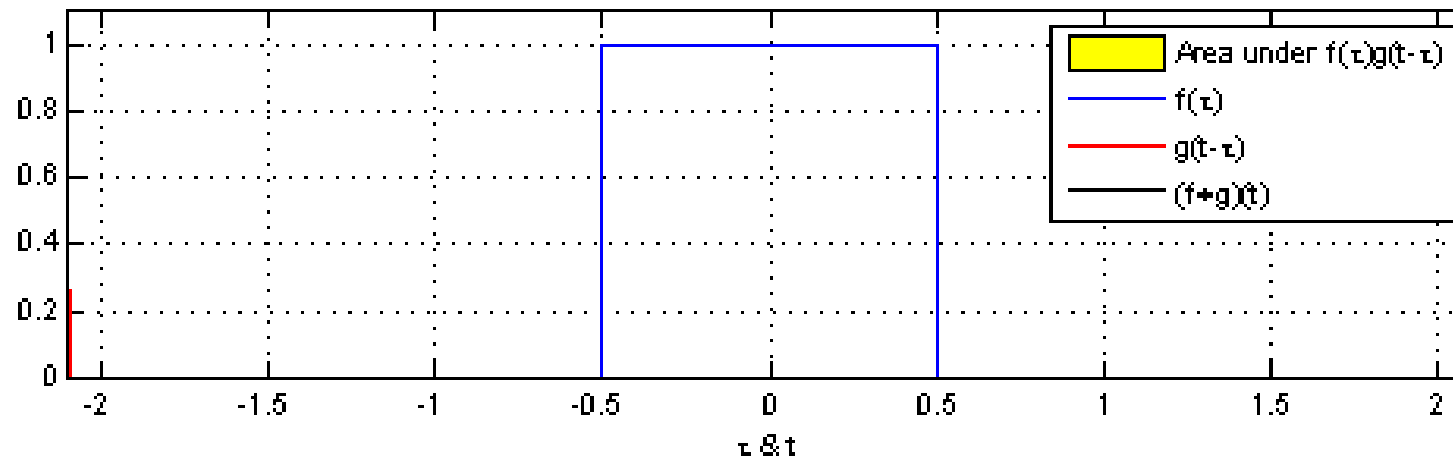
$$(f * g)(t) := \int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau.$$

# Convolution: formal definition

Convolution – Allows us to “combine” two functions expressing how the shape of one is modified by the other

- Used to characterize linear time-invariant (LTI) systems (impulse response)

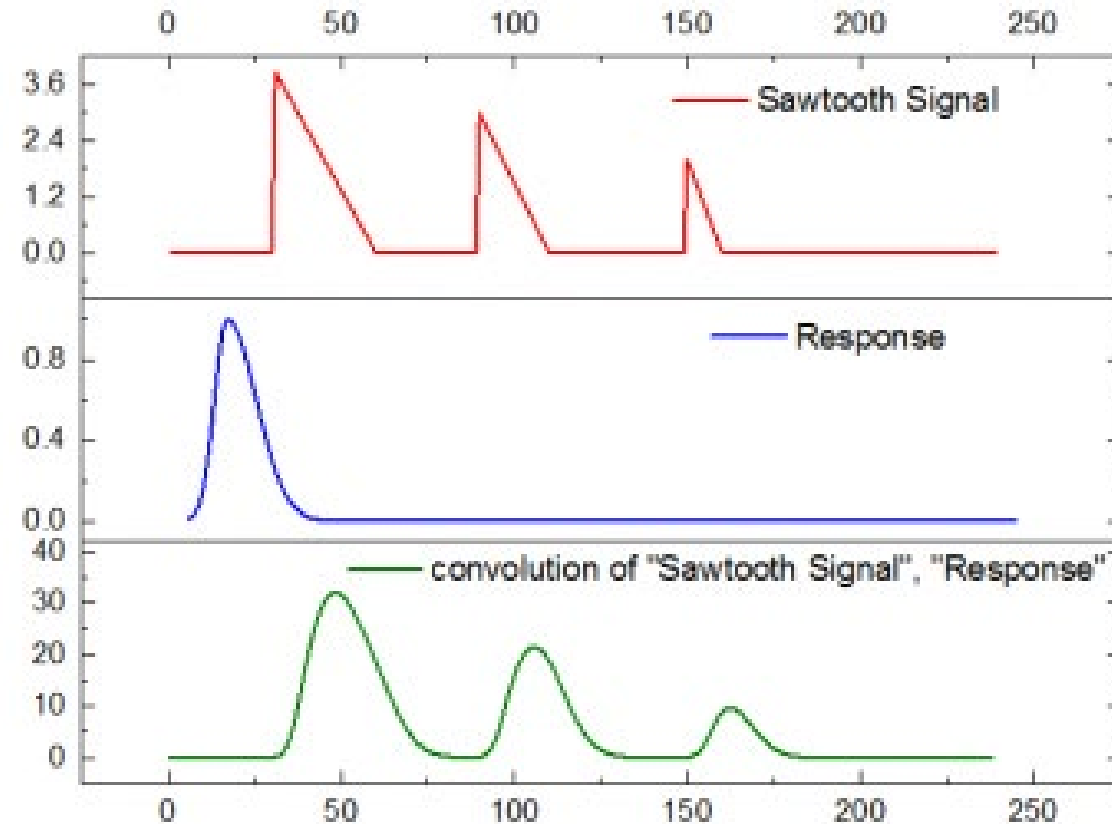
$$(f * g)(t) := \int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau.$$



# Convolution: formal definition

Convolution – Allows us to “combine” two functions expressing how the shape of one is modified by the other

- Used to characterize linear time-invariant (LTI) systems (impulse response)



# Convolution: formal definition

Convolution – Allows us to “combine” two functions expressing how the shape of one is modified by the other

- Used to characterize linear time-invariant (LTI) systems (impulse response)

Basic properties (there are more):

- **Commutativity**

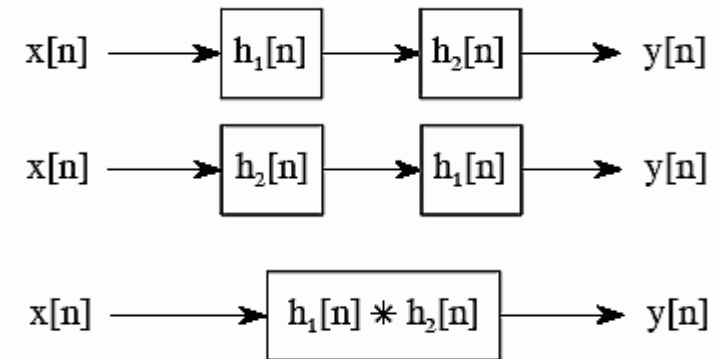
$$f * g = g * f$$

- **Associativity**

$$f * (g * h) = (f * g) * h$$

- **Distributivity**

$$f * (g + h) = (f * g) + (f * h)$$



# Convolution: formal definition

- 1D discrete convolution

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m]g[n - m],$$

$$(f * g)(t) := \int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau.$$

# Convolution: formal definition

- 1D discrete convolution

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m]g[n - m],$$

- 2D discrete convolution

$$(f * g)[n, m] = \sum_{j=-\infty}^{\infty} \sum_{i=-\infty}^{\infty} f[i, j] g[m - i, n - j]$$

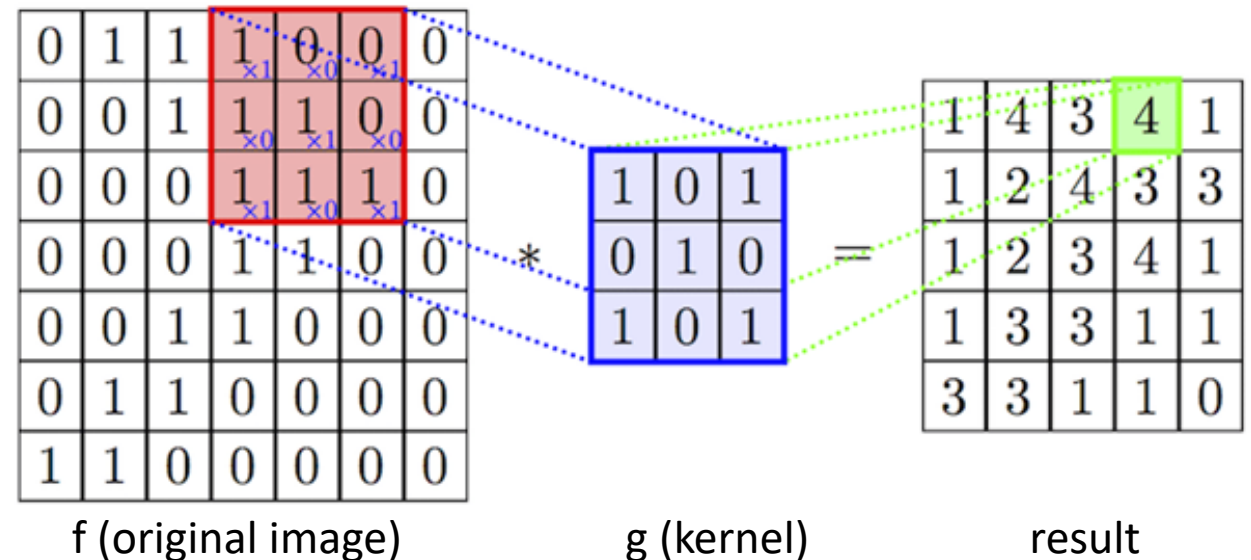
# Convolution: formal definition

- 1D discrete convolution

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m]g[n - m],$$

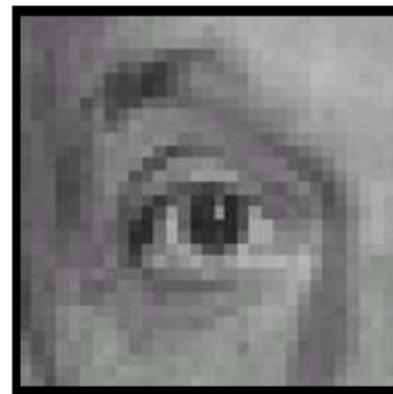
- 2D discrete convolution

$$(f * g)[n, m] = \sum_{j=-\infty}^{\infty} \sum_{i=-\infty}^{\infty} f[i, j] g[m - i, n - j]$$



# Blur as a convolution

- Replace each pixel by a linear combination of its neighbors.
  - Only depends on relative position of neighbors
- The prescription for the linear combination is called the “**convolution kernel**”.
  - Same kernel for all pixels



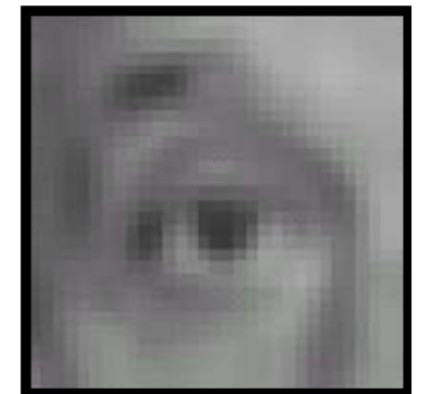
Original



$\frac{1}{9}$

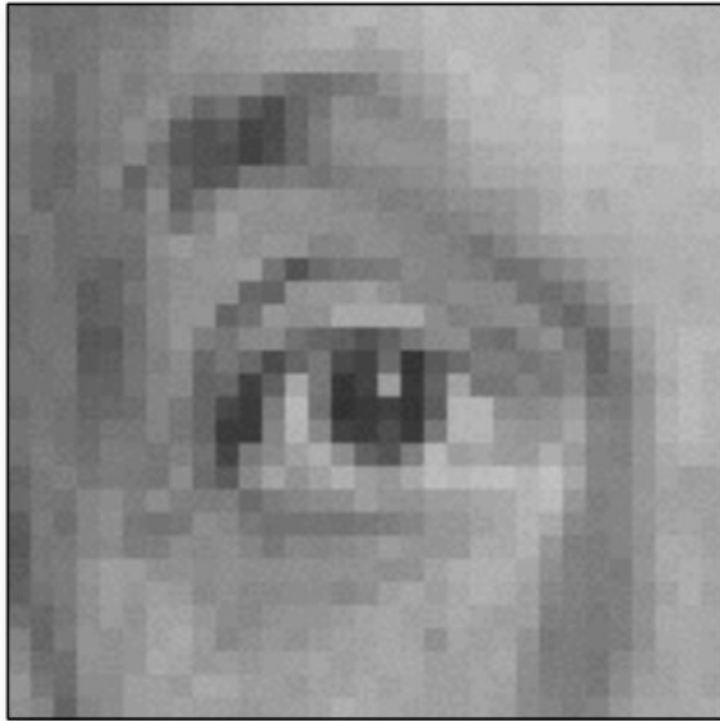
|   |   |   |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 1 |

=

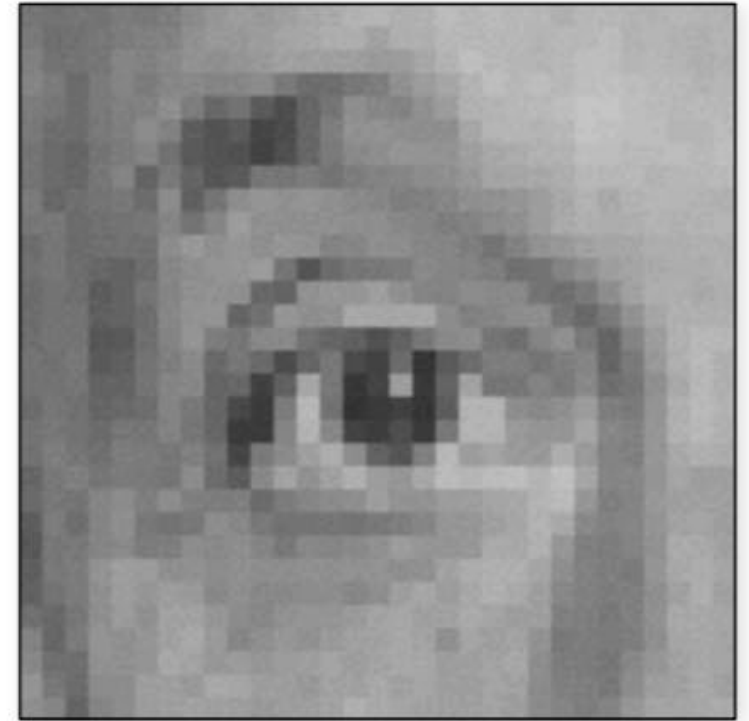
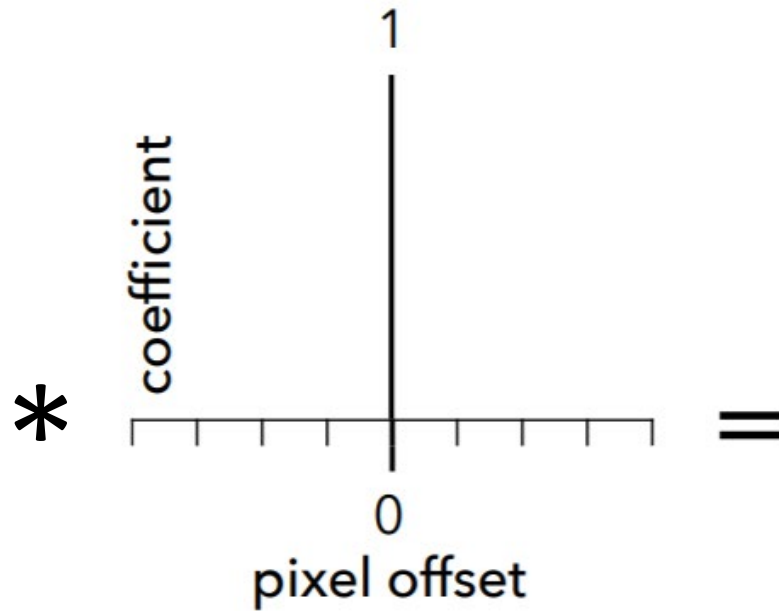


Blur (with a mean filter)

# Some examples

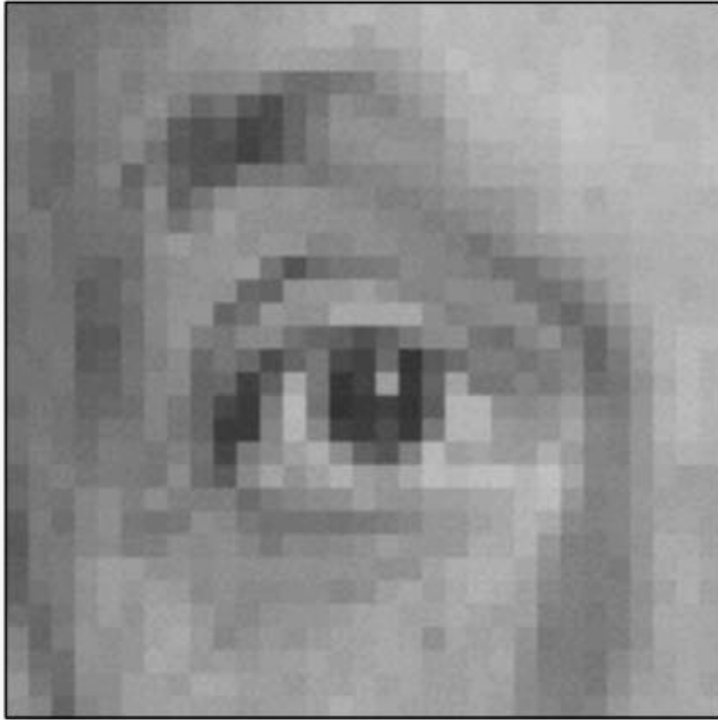


original

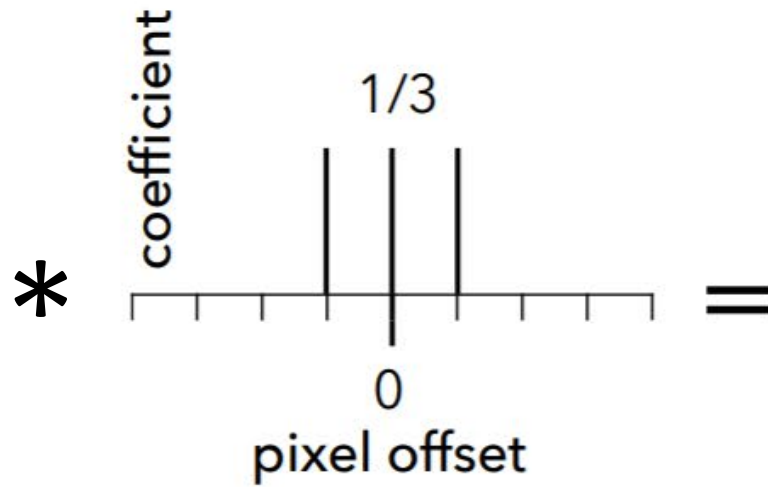


filtered  
(no change)

# Some examples

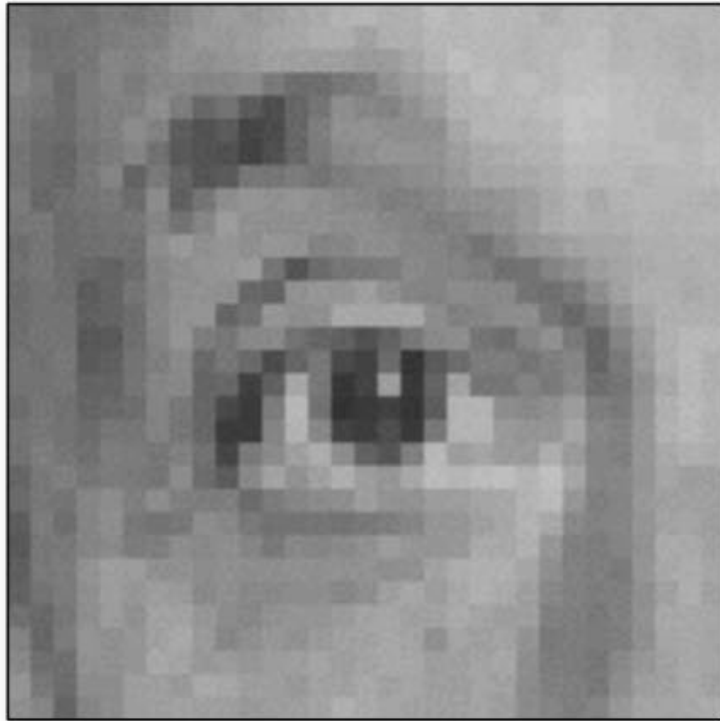


original

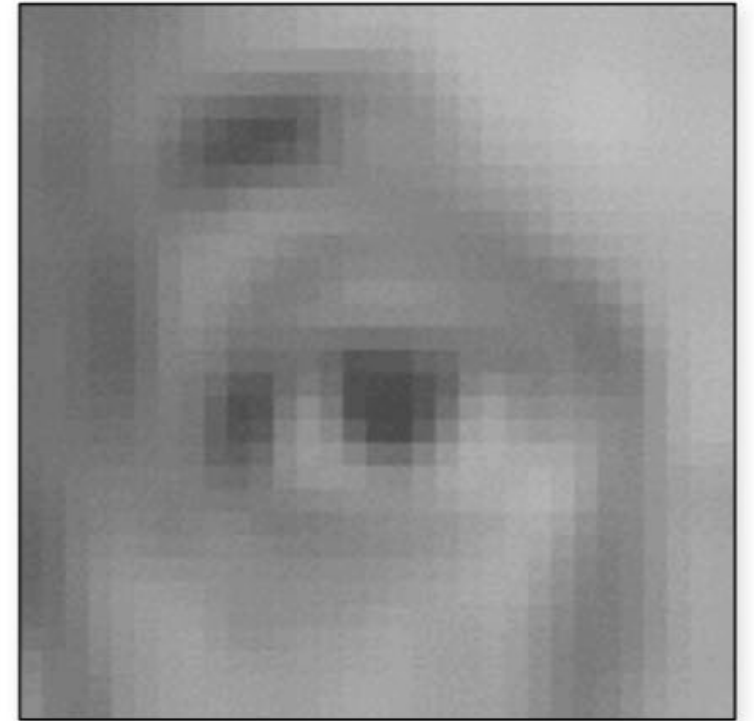
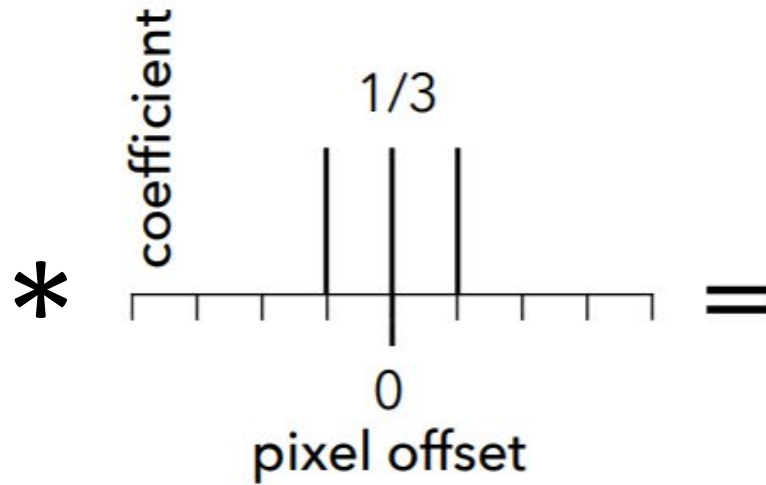


?

# Some examples

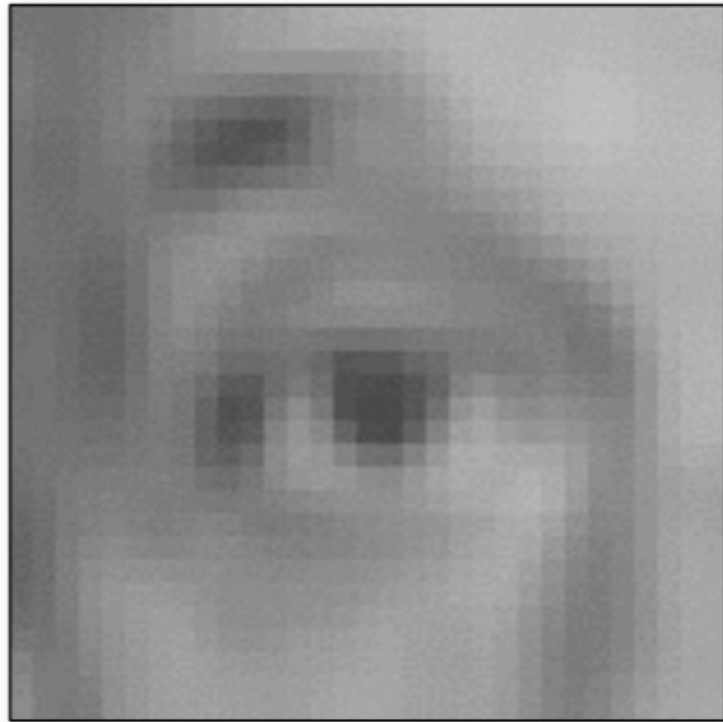


original

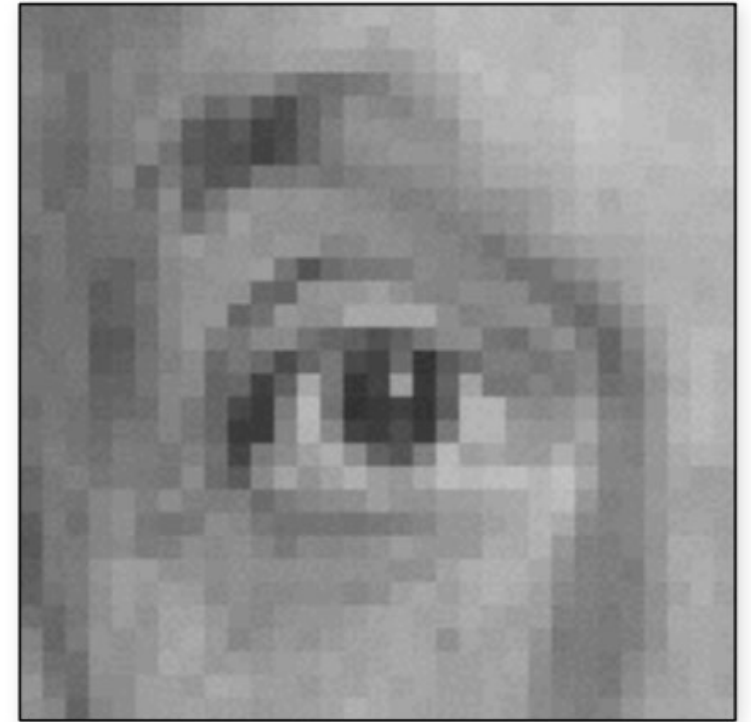
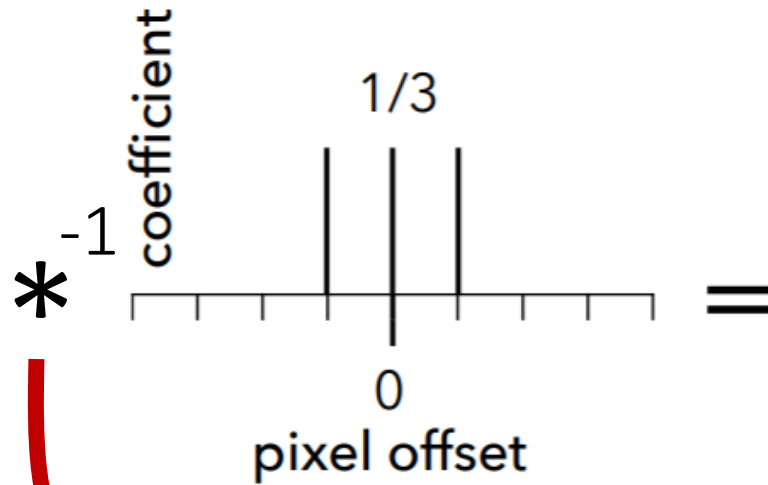


blurred  
(applied in both dimensions)

# ... and can we go the other way around?



blurred  
(applied in both dimensions)



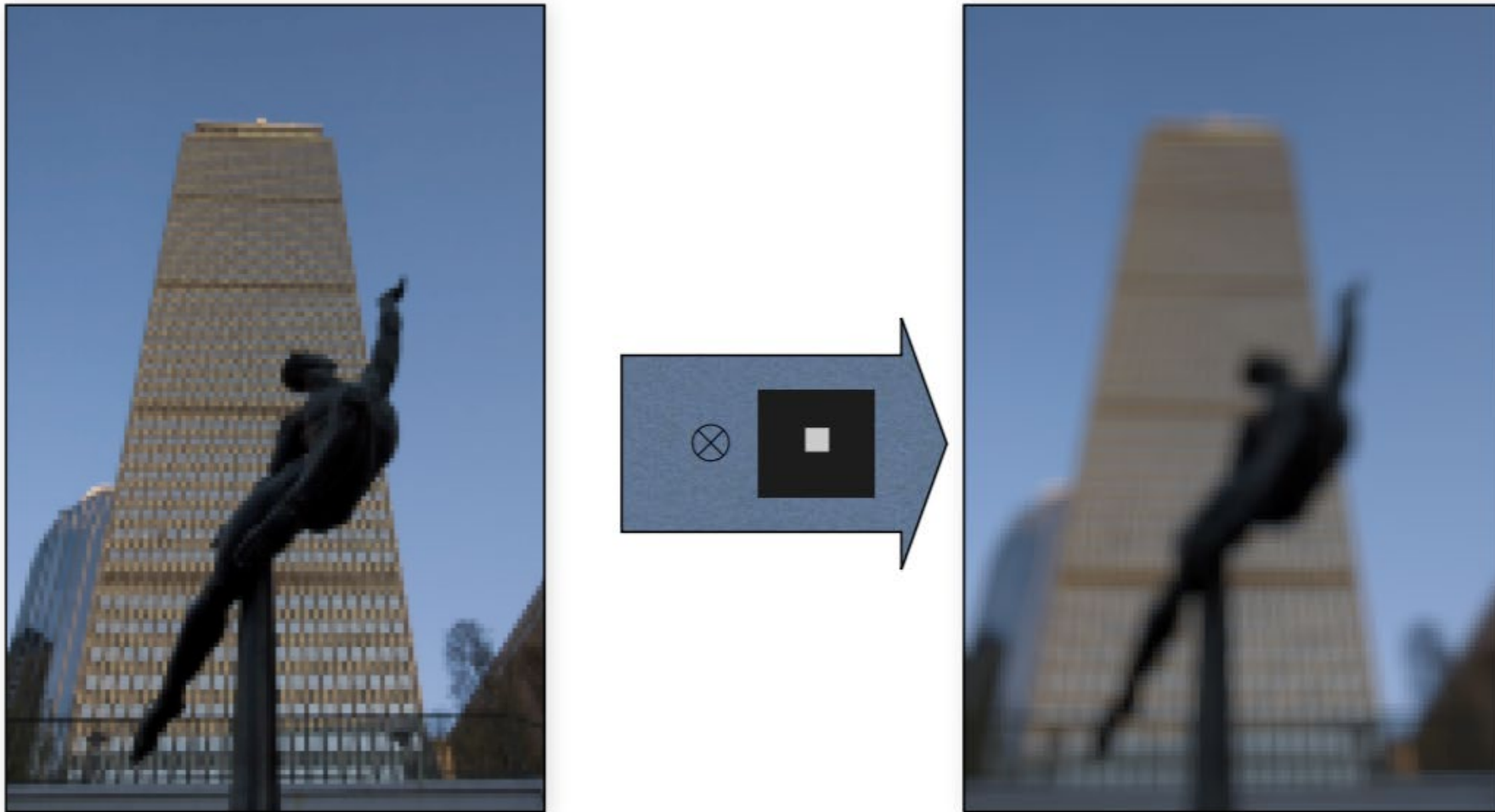
original

*Far from trivial*

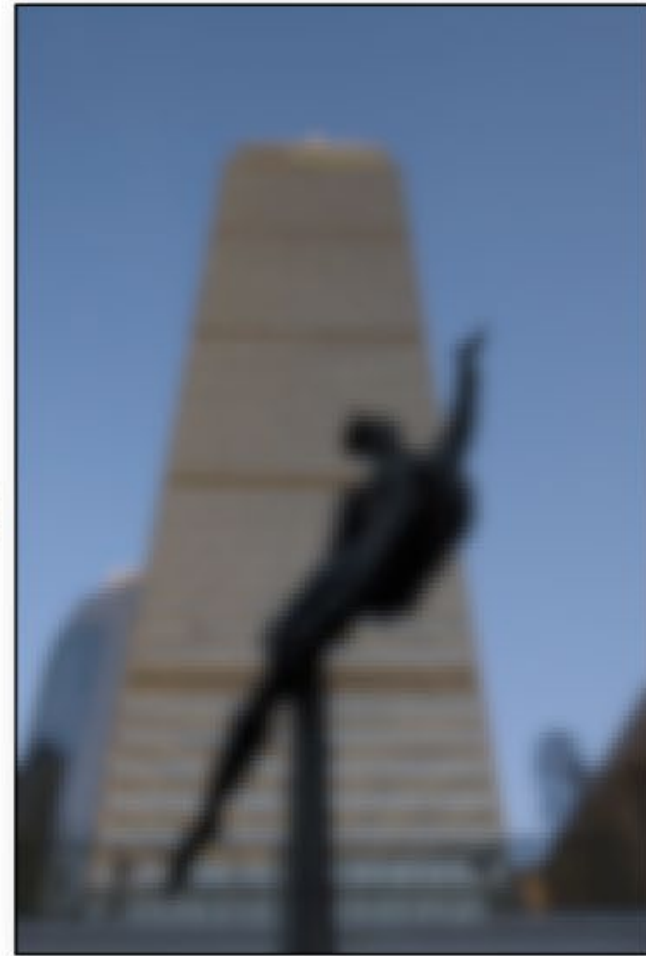
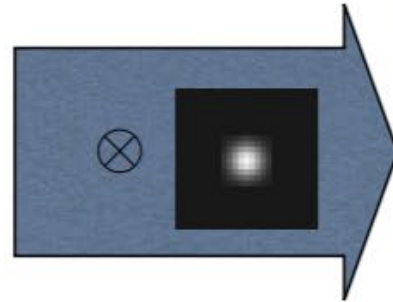
# Questions?

# A zoo of kernels

# Some examples – *Box Filter (low-pass filter)*



# Some examples – *Gaussian Filter (low-pass filter)*



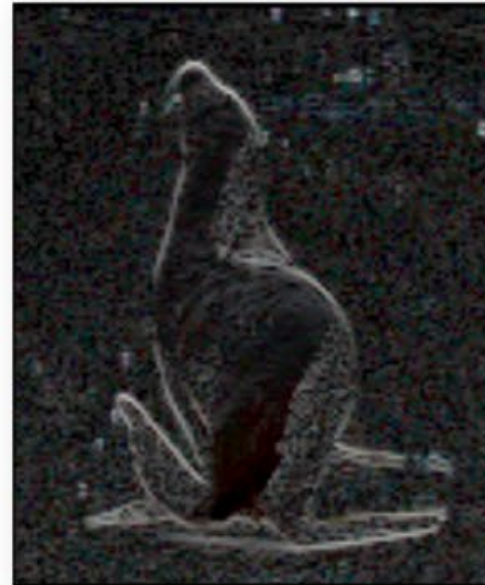
# Some examples – *Gradient Kernels (high-pass filter)*

horizontal gradient  $[-1, 1]$

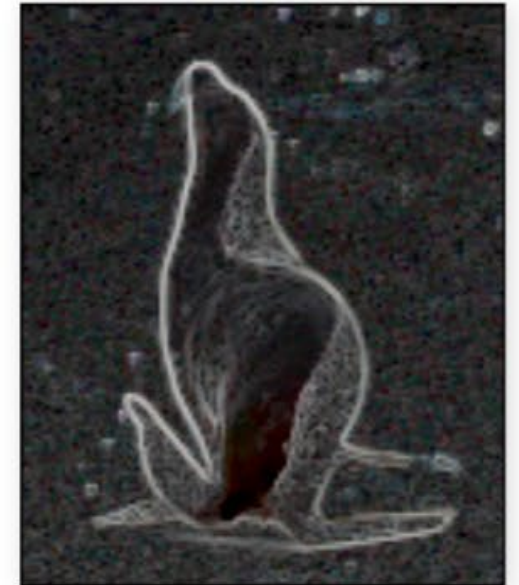
vertical gradient:  $[-1, 1]$



Horizontal gradient  
(absolute value)

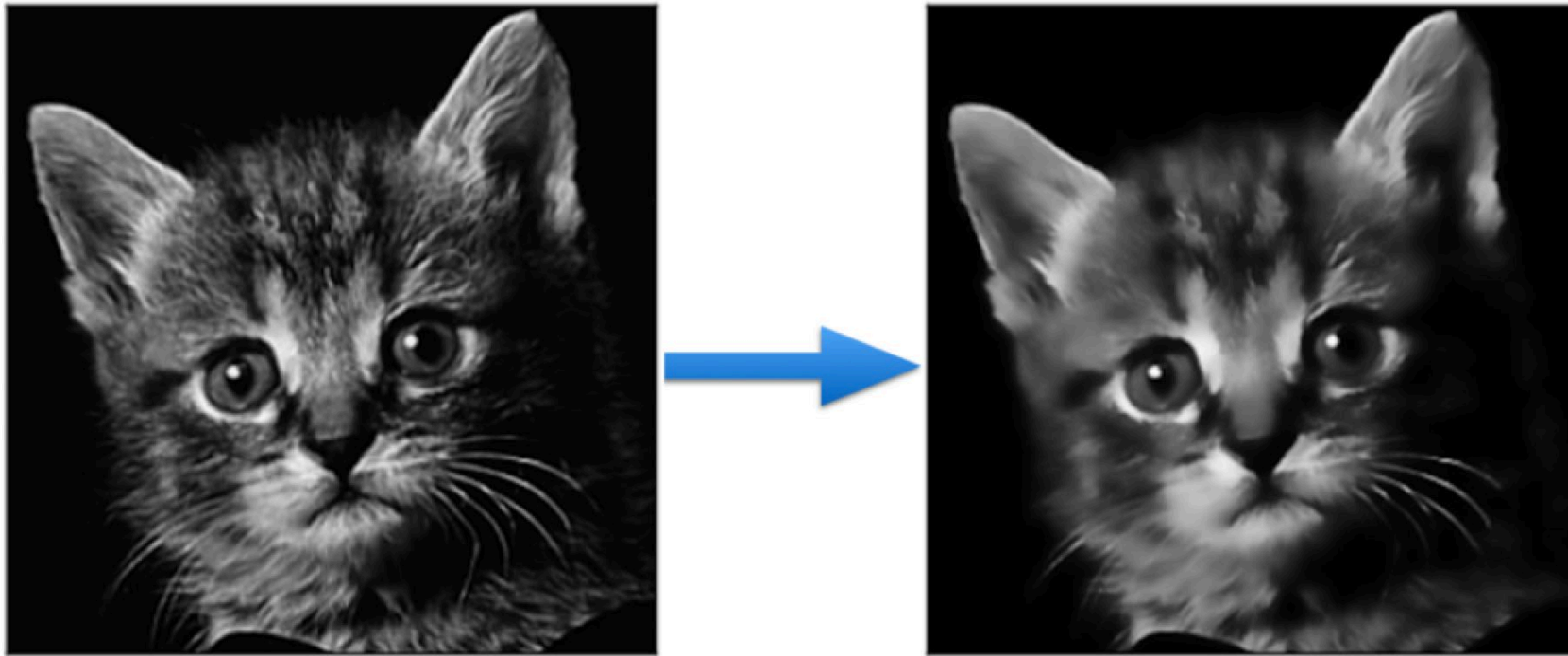
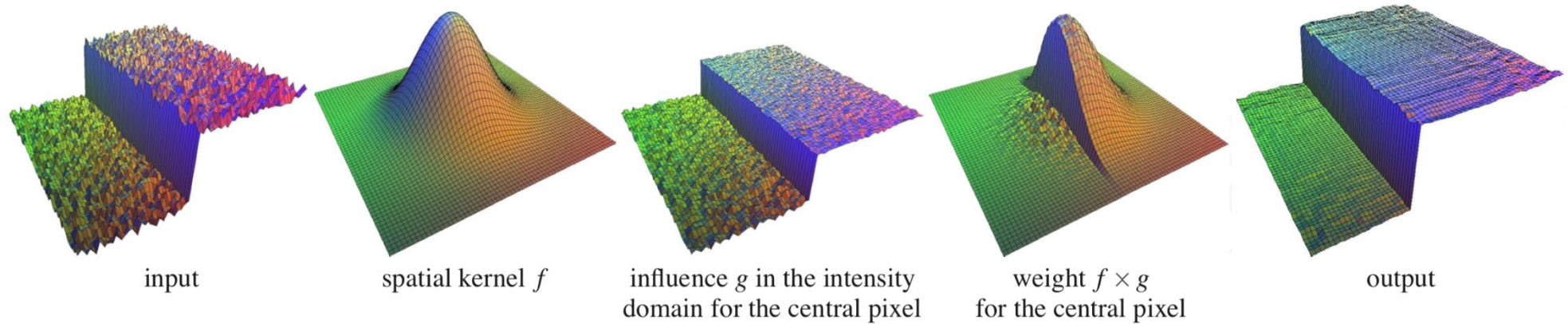


Vertical gradient  
(absolute value)



Gradient magnitude

# Some examples – *Edge-preserving kernels*



# A zoo of kernels

- Low-pass filters:
  - Box, Triangle, Gaussian, Lanczos, Mitchell, Epanechnikov, Perlin...
- High-pass filters:
  - Gradient, Sobel, Laplacian, Canny (edge detection), a family of unsharp filters...
- Edge preserving kernels:
  - Bilateral filter,...

You can play with some filters here: <https://setosa.io/ev/image-kernels/>

# Questions?

# So far...

- Blur as an inherent process of the image formation model.
  - Degrades visual information
- But also a powerful tool for image processing.
  - Useful for denoising, enhancement, artistic purposes...
- It bases on convolution!
  - But convolution might be expensive...  $O(N^2 \times K^2)$

# So far...

- **Point Spread Function (PSF)**: describes the response of an imaging system to a point source (system's impulse response).
- The degree of spreading (**blurring**) of the point source is a measure for the quality of an imaging system.
- The image captured by such imaging system will be affected by the PSF. This process is modeled by a **convolution** (that “combines” two functions expressing how the shape of one is modified by the other):

$$I_{captured} = I_{scene} * K_{PSF}$$

- Convolutions also model how images change when different kernels are applied, which is useful for **modeling common image processing operations** (gaussian filter, sobel filter...)
- But convolutions are expensive... **Fourier domain!**

# Fourier Analysis

# Fourier Series

A bold idea (1807):

*“**Any** univariate function can be rewritten as a weighted sum of sines and cosines of different frequencies. “*



Jean Baptiste Joseph Fourier (1768-1830)

# Fourier Series

A bold idea (1807):

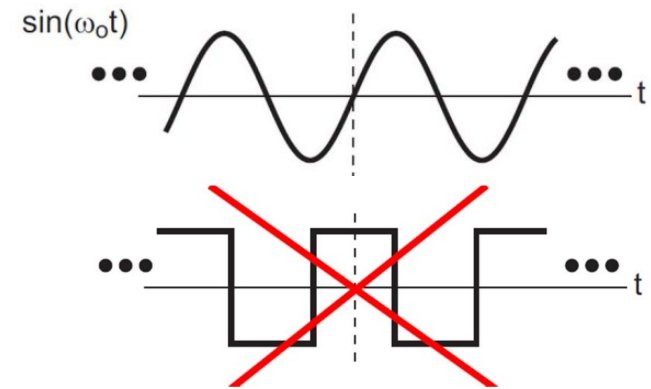
*“**Any** univariate function can be rewritten as a weighted sum of sines and cosines of different frequencies. ”*

Our building block:

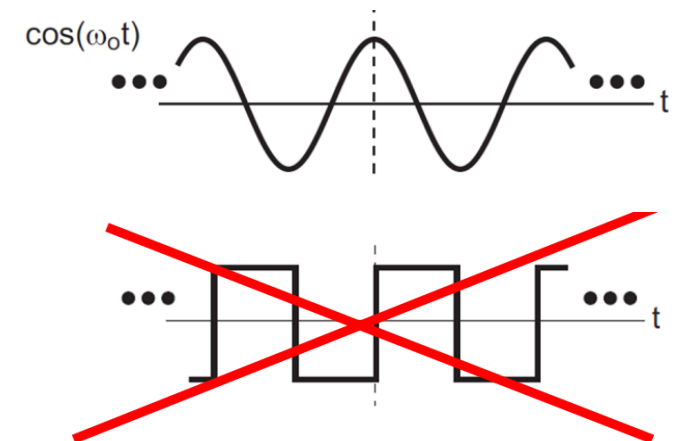
$$A \sin(\omega t) + B \cos(\omega t)$$

Add enough of them to get any signal  $f(t)$  you want!

Sines are odd / anti-symmetric:  
Cannot create even functions



Cosines are even / symmetric:  
Cannot create odd functions



# Fourier Series

A bold idea (1807):

*“**Any** univariate function can be rewritten as a weighted sum of sines and cosines of different frequencies. “*

Our building block:

$$A \sin(\omega t) + B \cos(\omega t)$$

Add enough of them to get any signal  $f(t)$  you want!

# Fourier Series

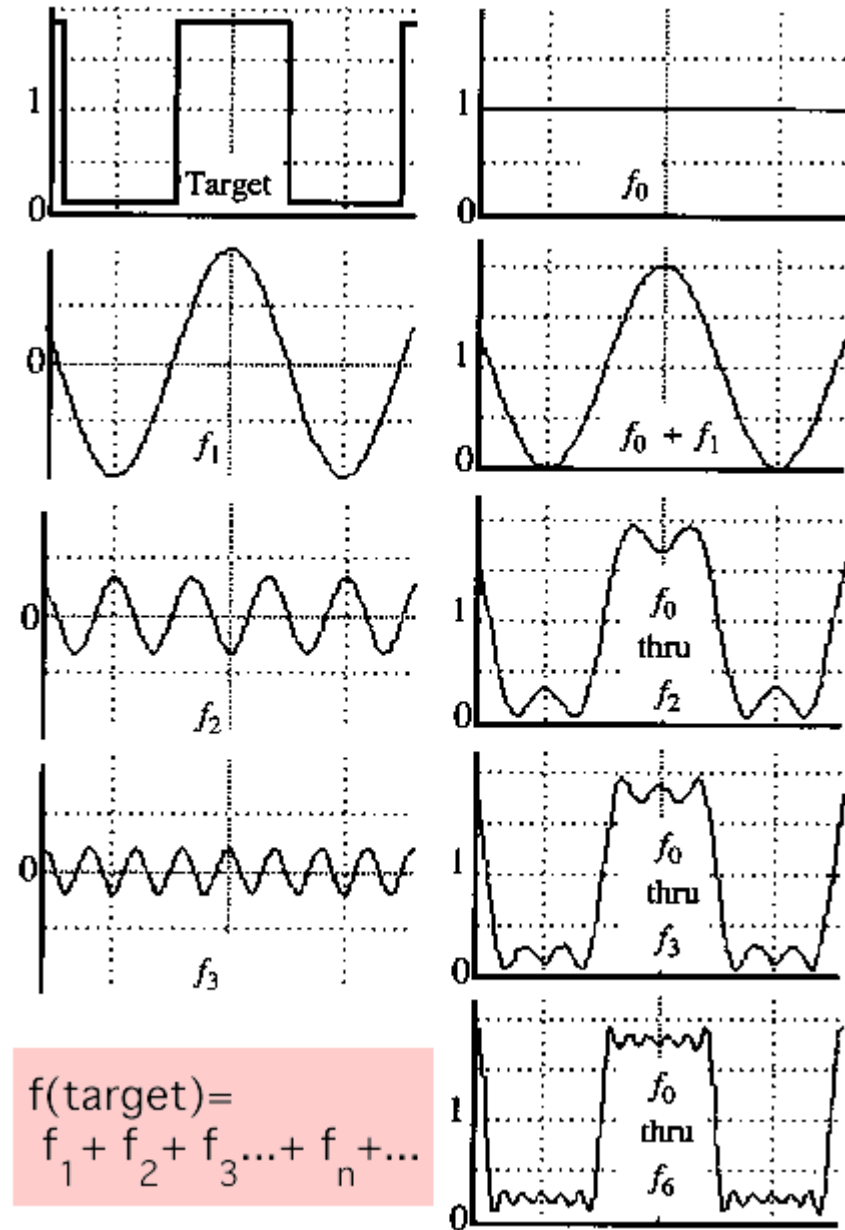
A bold idea (1807):

*“Any univariate function can be rewritten as a weighted sum of sines and cosines of different frequencies.”*

Our building block:

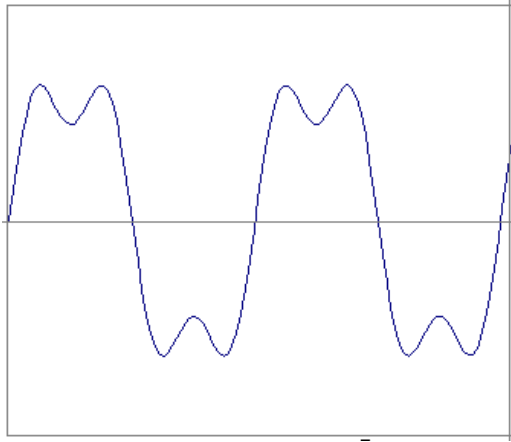
$$A \sin(\omega t) + B \cos(\omega t)$$

Add enough of them to get any signal  $f(t)$  you want!



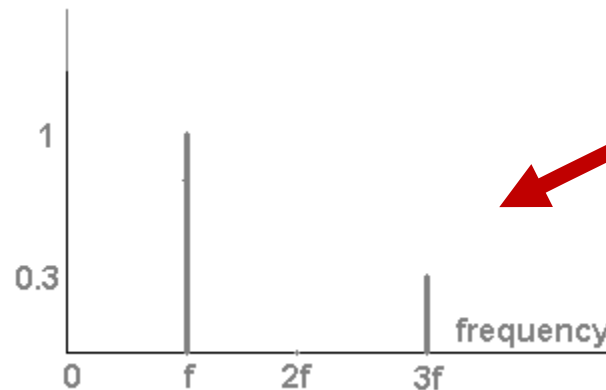
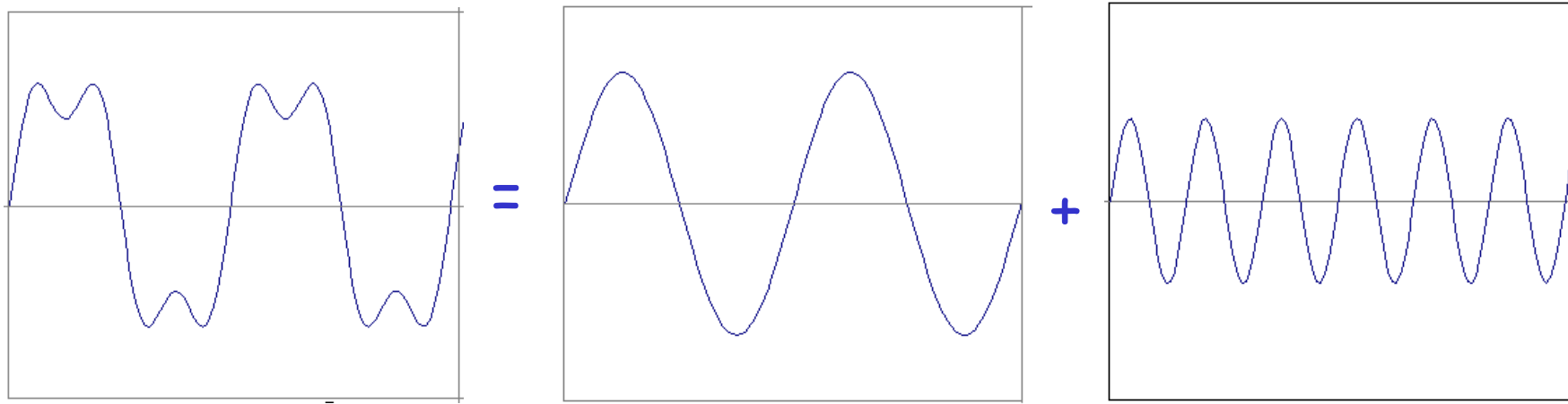
# Fourier Series – *One example*

- $f(t) = \sin(2pf t) + (1/3)\sin(2p(3f) t)$



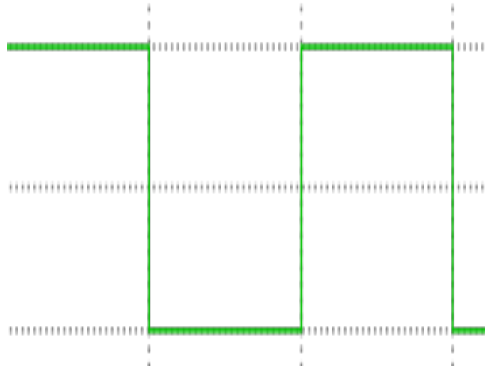
# Fourier Series – *One example*

- $f(t) = \sin(2pf t) + (1/3)\sin(2p(3f) t)$

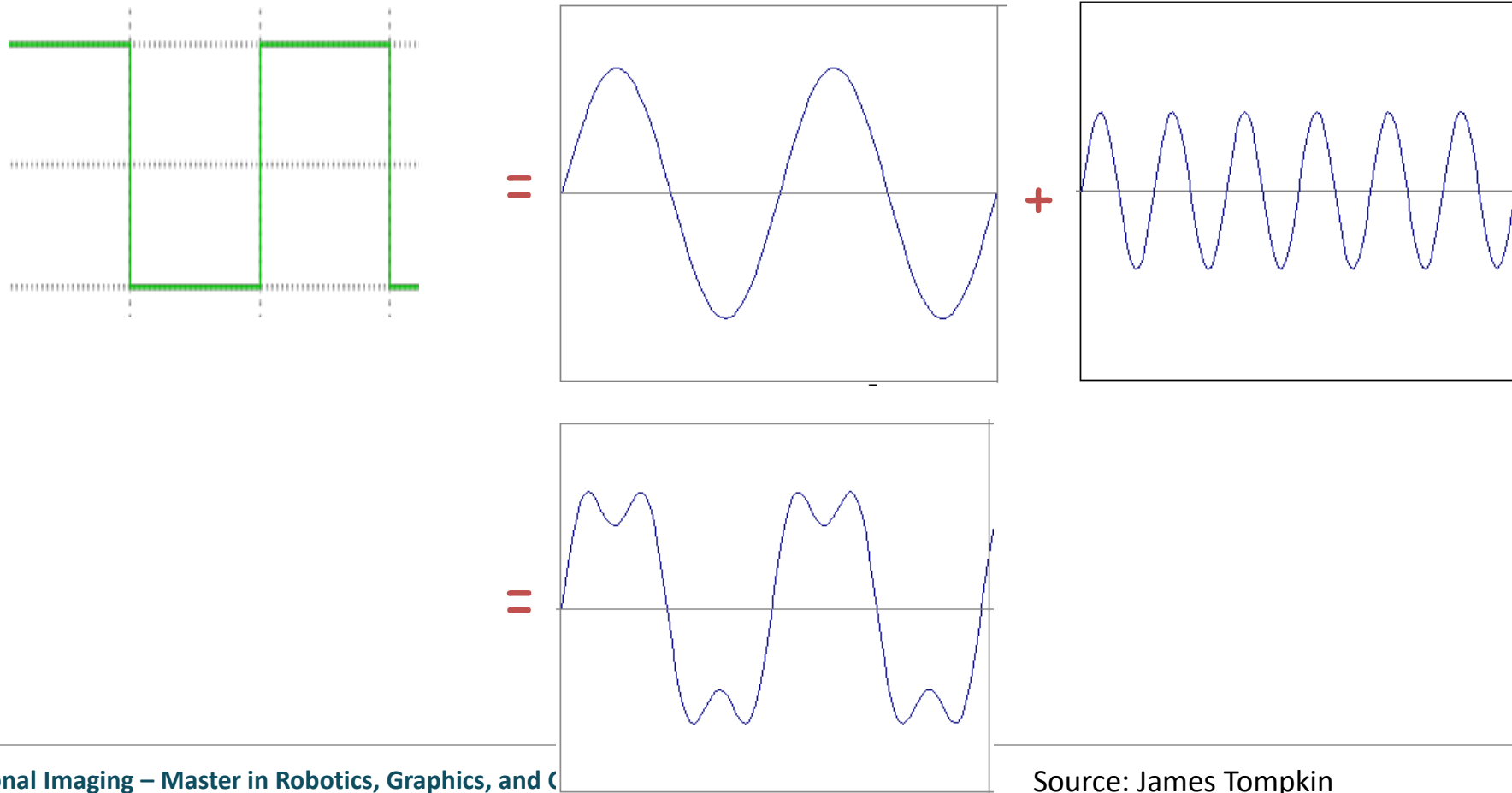


Fourier Spectra  
of  $f(t)$

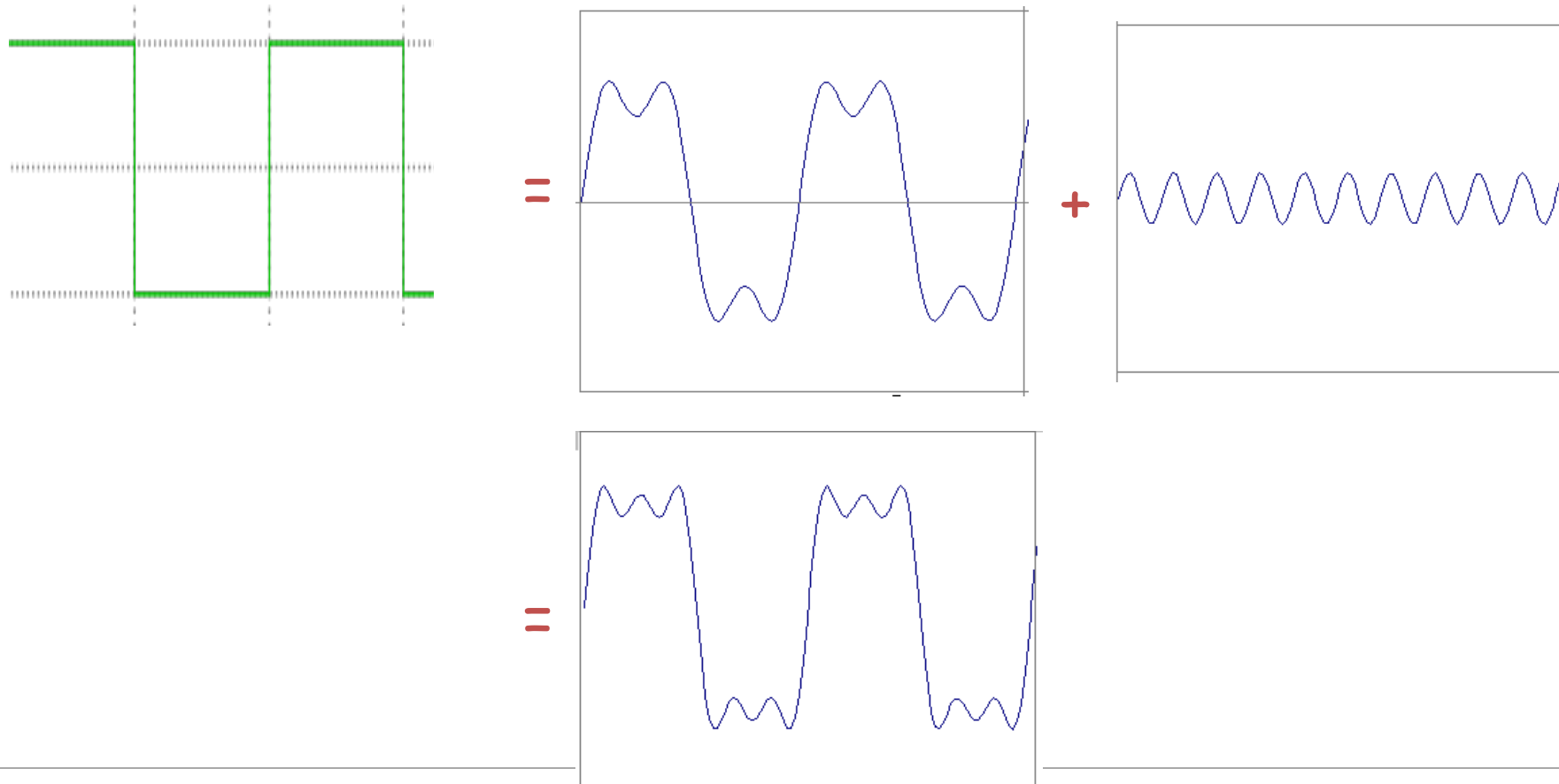
# Fourier Series – *Square wave spectra*



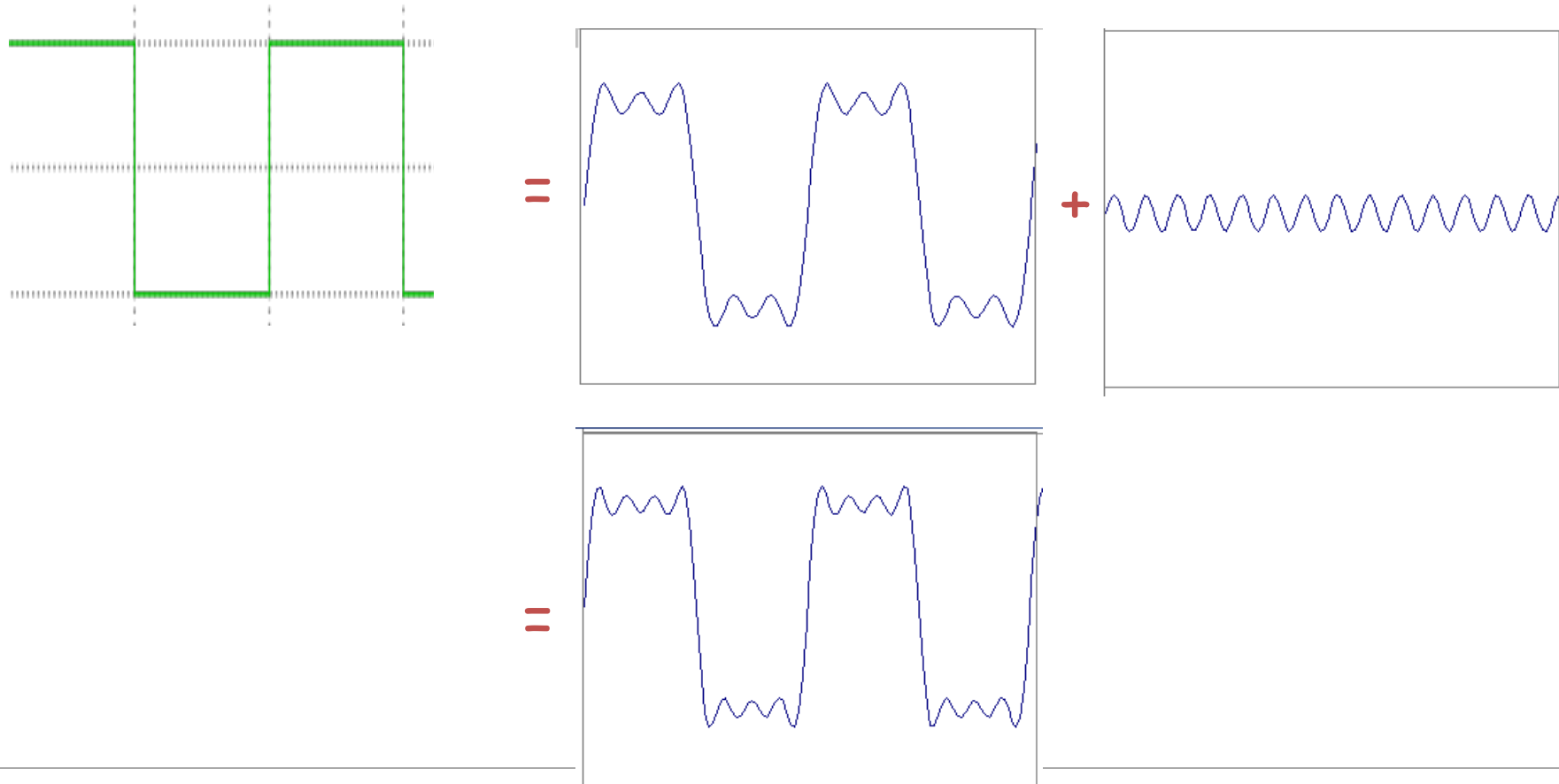
# Fourier Series – *Square wave spectra*



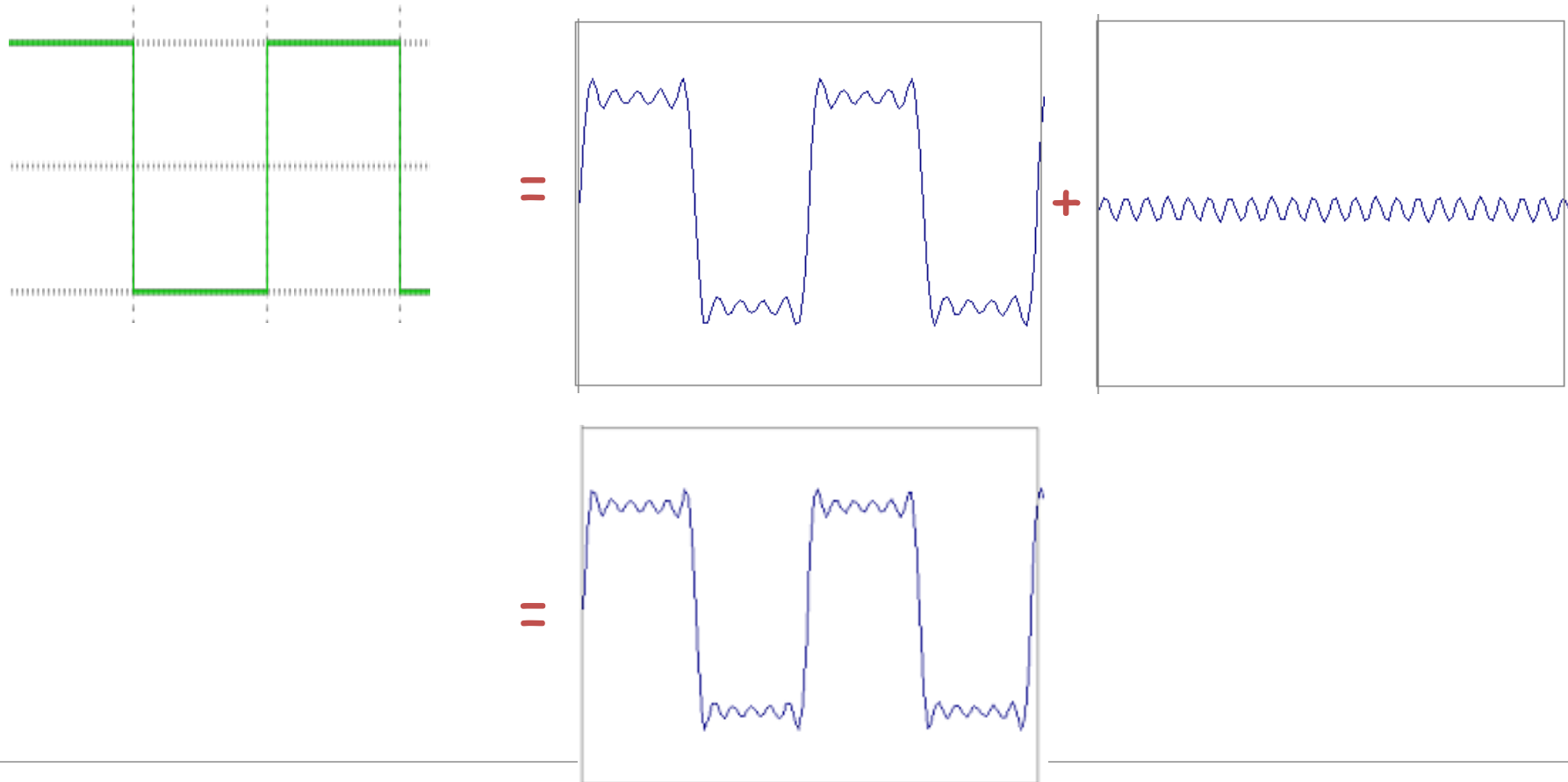
# Fourier Series – *Square wave spectra*



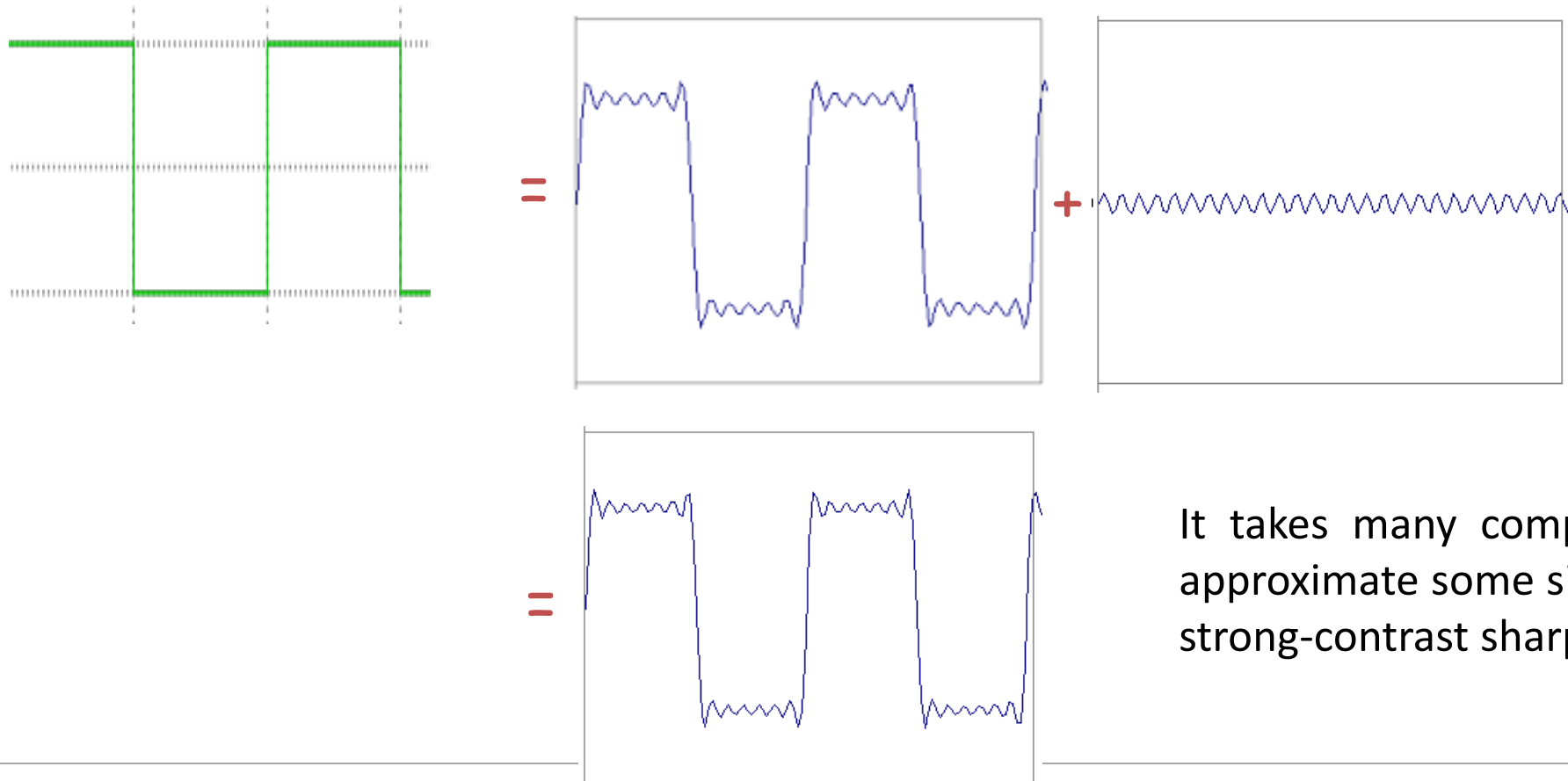
# Fourier Series – *Square wave spectra*



# Fourier Series – *Square wave spectra*

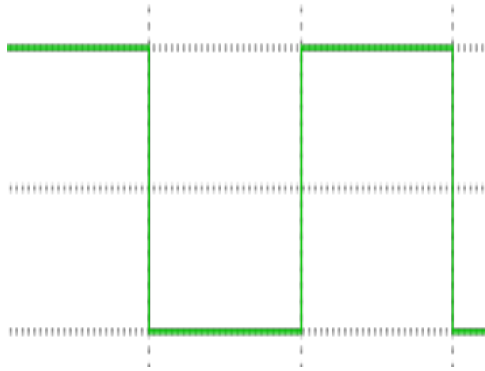


# Fourier Series – *Square wave spectra*

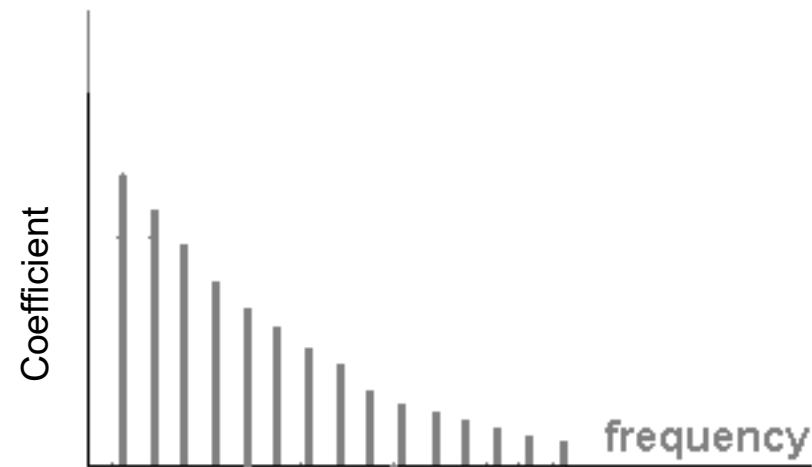


It takes many components to approximate some signals (e.g., strong-contrast sharp edges)!

# Fourier Series – *Square wave spectra*



$$= A \sum_{f=1}^{\infty} \frac{1}{f} \sin(2\pi f t)$$



# The Fourier Transform

# Fourier Series → Fourier Transform

The Fourier transform is more general and can be applied to non-periodic as well as periodic signals. It's used to analyze signals that do not necessarily repeat.

$$F(w) = \int_{-\infty}^{\infty} f(t) e^{-i2\pi f t} dt$$

# Fourier Series → Fourier Transform

The Fourier transform is more general and can be applied to non-periodic as well as periodic signals. It's used to analyze signals that do not necessarily repeat.

$$F(w) = \int_{-\infty}^{\infty} f(t)e^{-i2\pi ft} dt$$

Why complex numbers?

- Real signals can be represented as a combination of sine and cosine functions at different frequencies.
- Sine and cosine functions are complex exponentials in Euler's formula.
- The complex numbers in the Fourier transform represent the **amplitudes** (strength) and **phases** (timing) of these sinusoidal components.

# Fourier Series → Fourier Transform

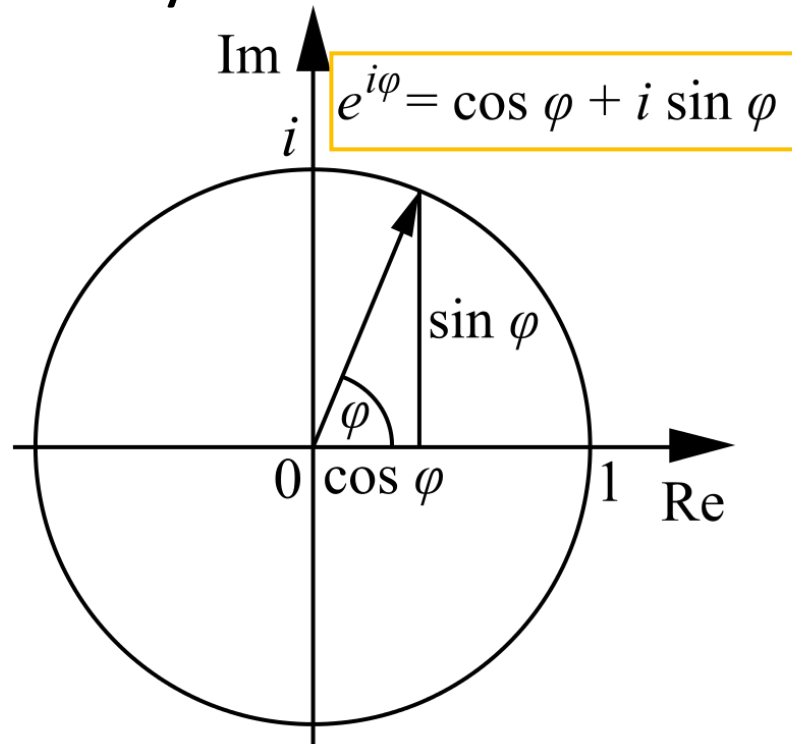
The Fourier transform is more general and can be applied to non-periodic as well as periodic signals. It's used to analyze signals that do not necessarily repeat.

$$F(w) = \int_{-\infty}^{\infty} f(t) e^{-i2\pi f t} dt$$

$$F(w) = \int_{-\infty}^{\infty} f(t) (\cos(2\pi f t) - i \sin(2\pi f t)) dt$$

# Fourier Transform

- Stores the **amplitude** and **phase** at each **frequency**:
  - For mathematical convenience, this is often notated in terms of real and complex numbers
  - Related by Euler's formula



Amplitude encodes how much signal there is at a particular frequency:

$$A = \pm \sqrt{\text{Re}(\varphi)^2 + \text{Im}(\varphi)^2}$$

Phase encodes spatial information (indirectly):

$$\phi = \tan^{-1} \frac{\text{Im}(\varphi)}{\text{Re}(\varphi)}$$

# Fourier Transform

$$F(f) = \int_{-\infty}^{\infty} f(t) e^{-i2\pi f t} dt$$

Further explanations and intuitive interpretation: <https://sites.northwestern.edu/elannesscohn/2019/07/30/developing-an-intuition-for-fourier-transforms/>

# Fourier Transform

$$F(f) = \int_{-\infty}^{\infty} f(t) e^{-i2\pi f t} dt$$

$$f(t) = \int_{-\infty}^{\infty} F(f) e^{i2\pi f t} df$$

Further explanations and intuitive interpretation: <https://sites.northwestern.edu/elannesscohn/2019/07/30/developing-an-intuition-for-fourier-transforms/>

# Fourier Transform

$$F(f) = \int_{-\infty}^{\infty} f(t) e^{-i2\pi f t} dt$$

$$f(t) = \int_{-\infty}^{\infty} F(f) e^{i2\pi f t} df$$

$$X(k) = \sum_{n=0}^{N-1} x[n] e^{-i2\pi \frac{k}{N} n}$$

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{i2\pi \frac{k}{N} n}$$

Further explanations and intuitive interpretation: <https://sites.northwestern.edu/elannesscohn/2019/07/30/developing-an-intuition-for-fourier-transforms/>

# Fourier Transform – *Properties*

- Linearity  $F[ax(t) + by(t)] = a F[x(t)] + b F[y(t)]$
- Fourier transform of a real signal is symmetric about the origin
- The energy of the signal is the same as the energy of its Fourier transform

# Fourier Transform – *The Convolution Theorem*

- The Fourier transform of the convolution of two functions is the product of their Fourier transforms

$$F[g * h] = F[g]F[h]$$

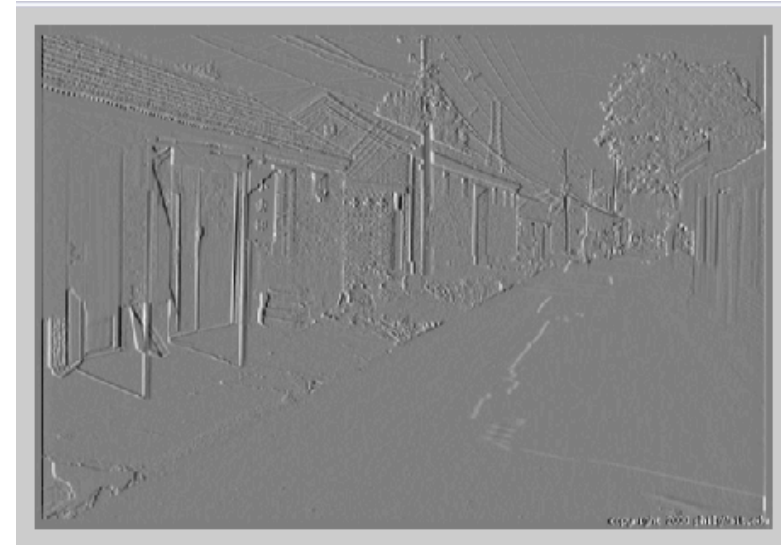
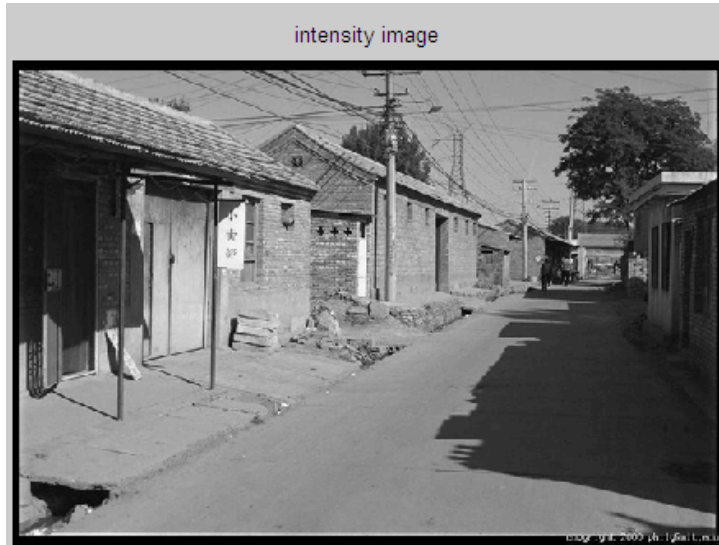
- **Convolution** in spatial domain is equivalent to **multiplication** in frequency domain!

$$g * h = F^{-1}[F[g]F[h]]$$

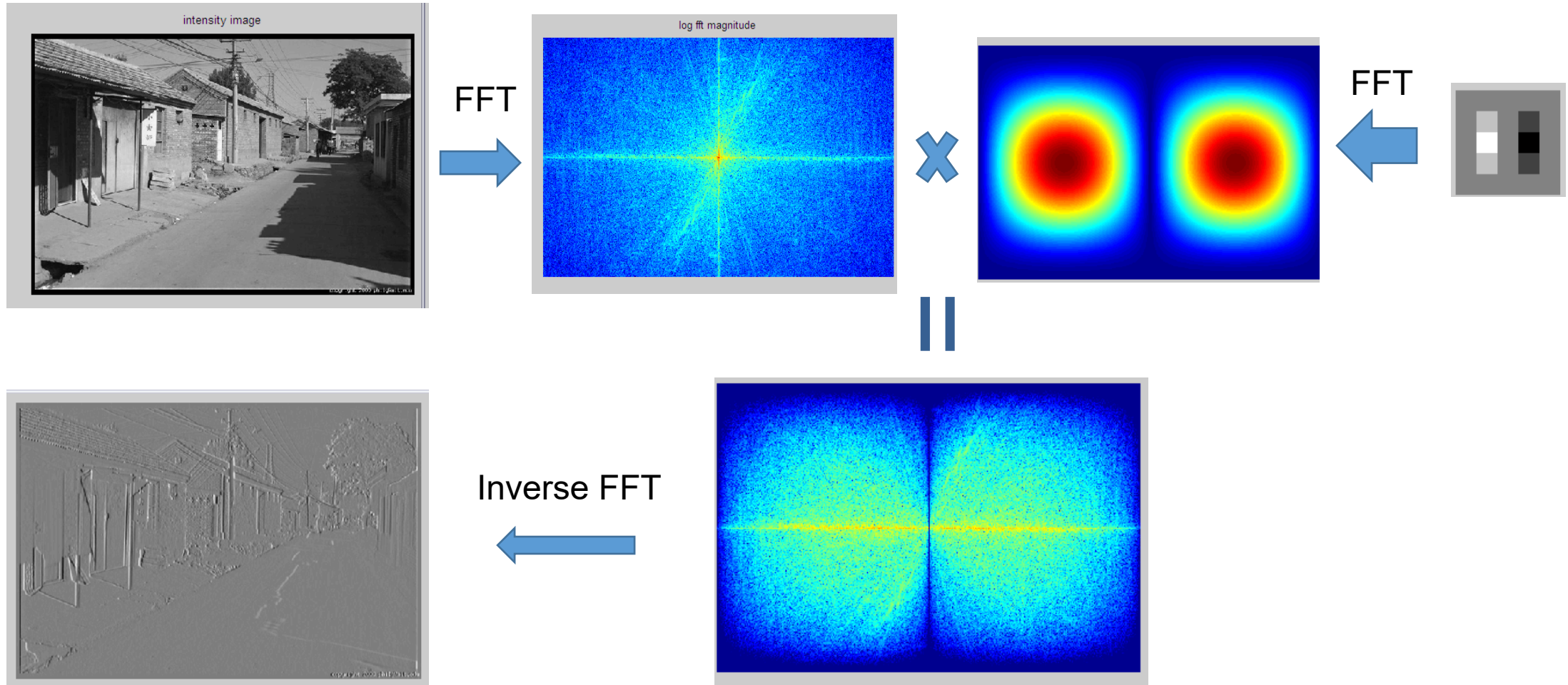
# Fourier Transform – *Why do we care?*

- Makes convolution cheaper (a multiplication)
- Can be used as alternative domain for sparsity (useful for e.g., compression)
- Image processing “language”, used in a wide range of applications: image analysis, filtering, edge detection, reconstruction, compression...

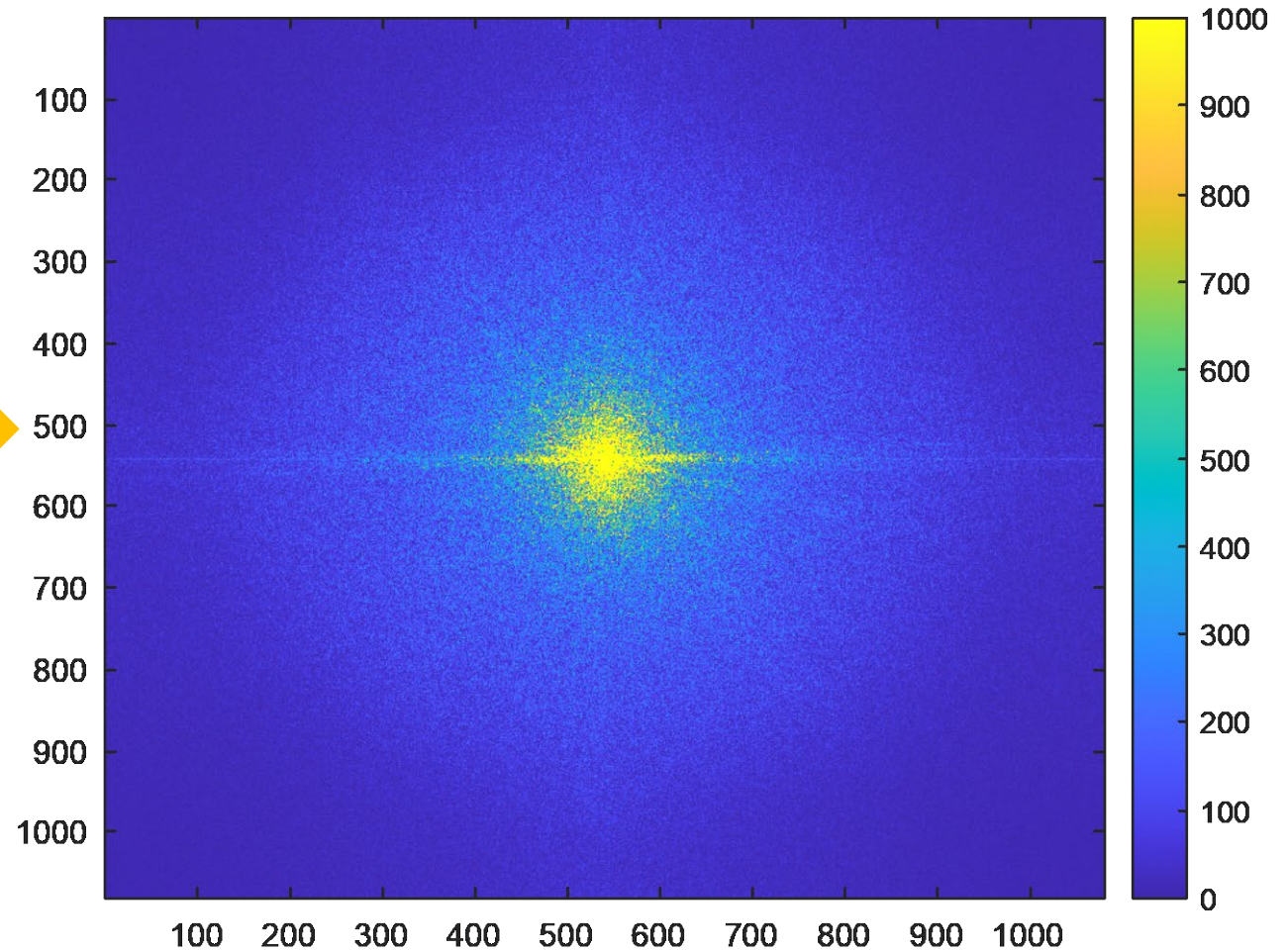
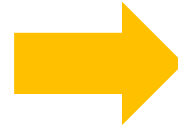
# Fourier Transform – *Filtering in spatial domain*



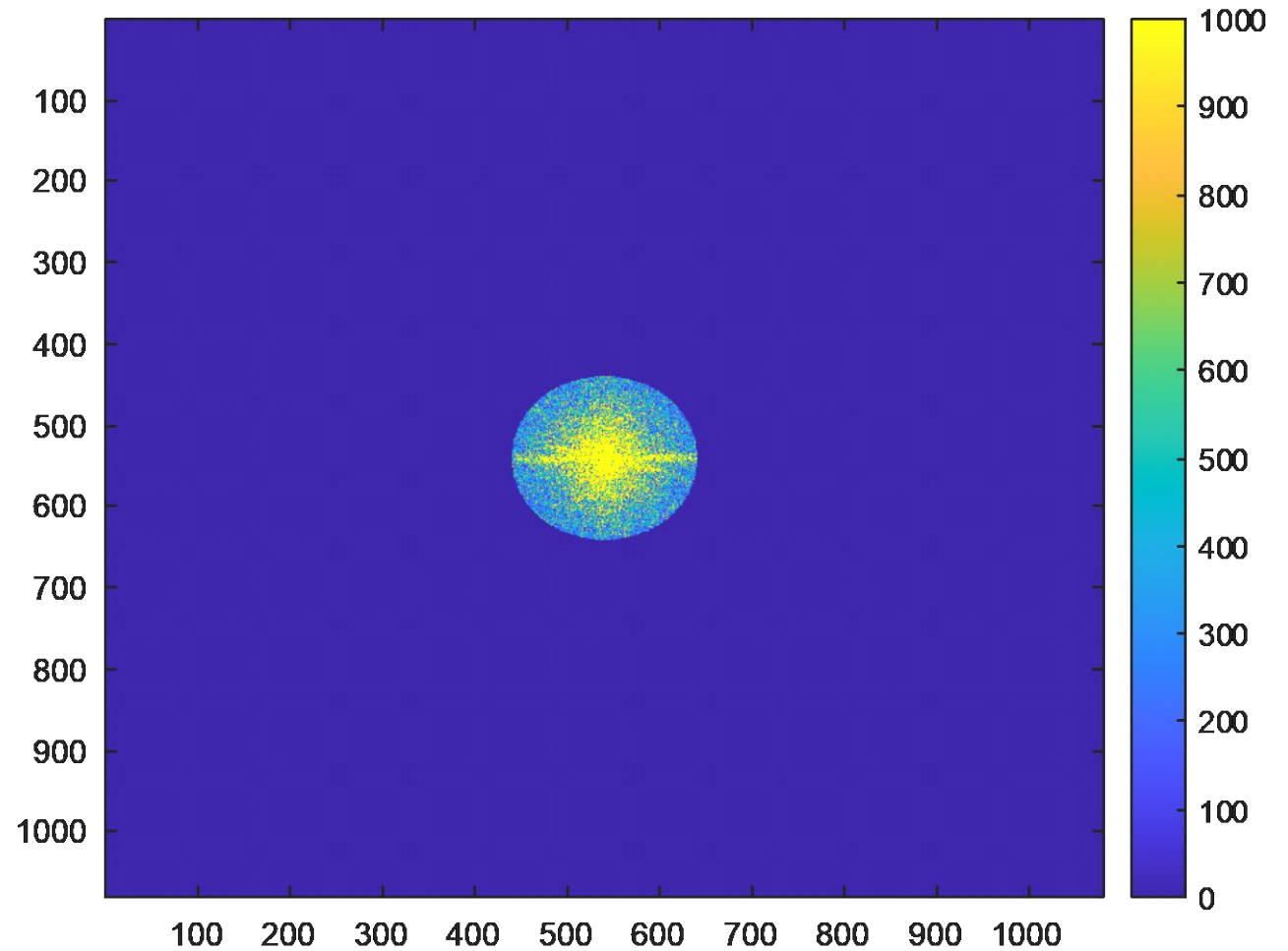
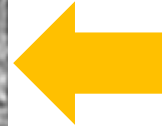
# Fourier Transform – *Filtering in frequency domain*



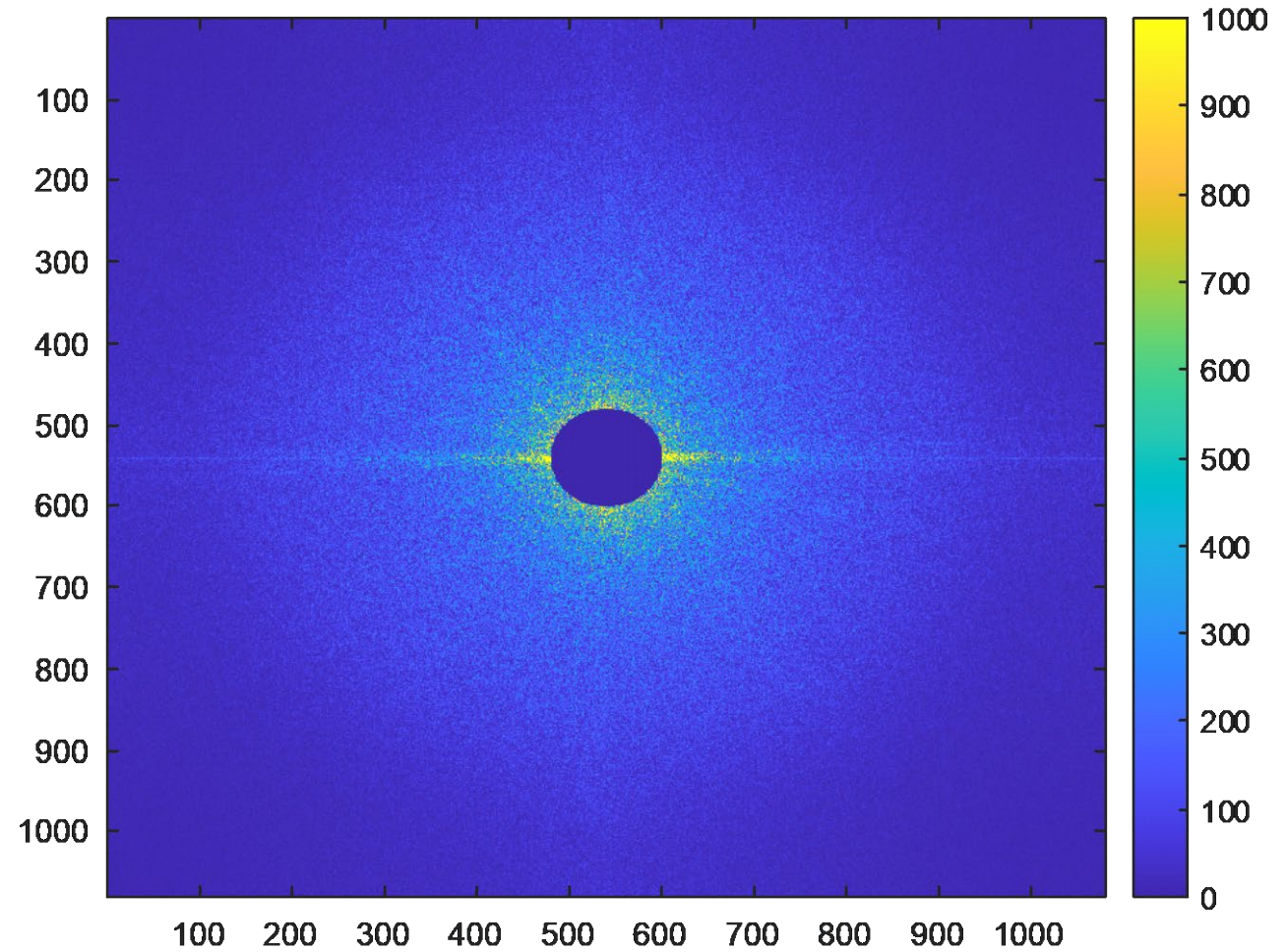
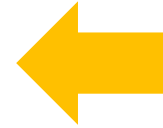
# Fourier Transform – *filtering*



# Fourier Transform – *filtering*



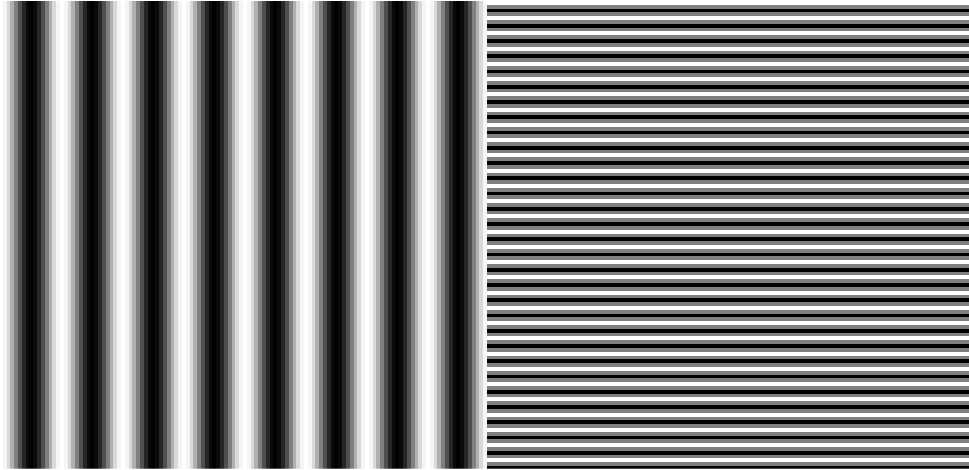
# Fourier Transform – *filtering*



# Fourier Transform – *Interpreting the freq. domain*

horizontal cos (low-f.)

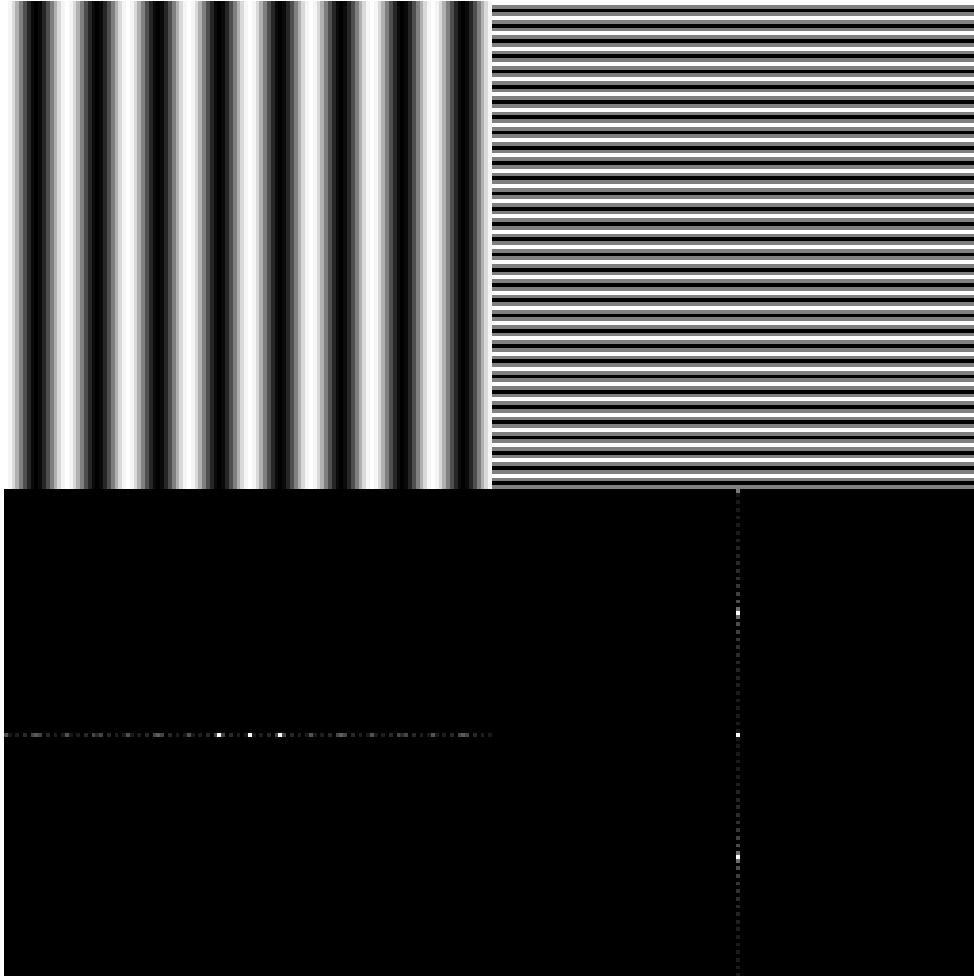
vertical cos (high-f.)



# Fourier Transform – *Interpreting the freq. domain*

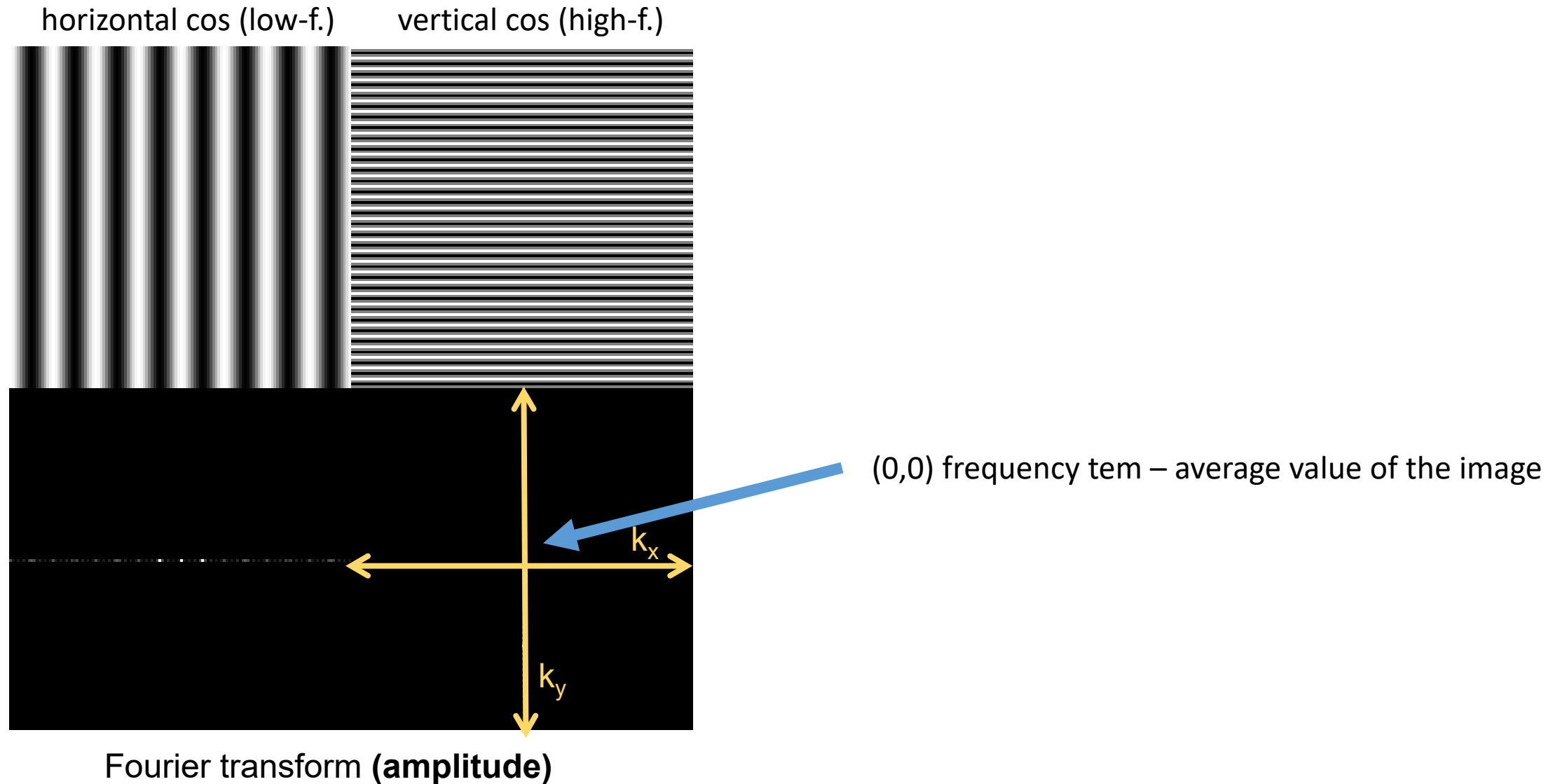
horizontal cos (low-f.)

vertical cos (high-f.)



Fourier transform (**amplitude**)

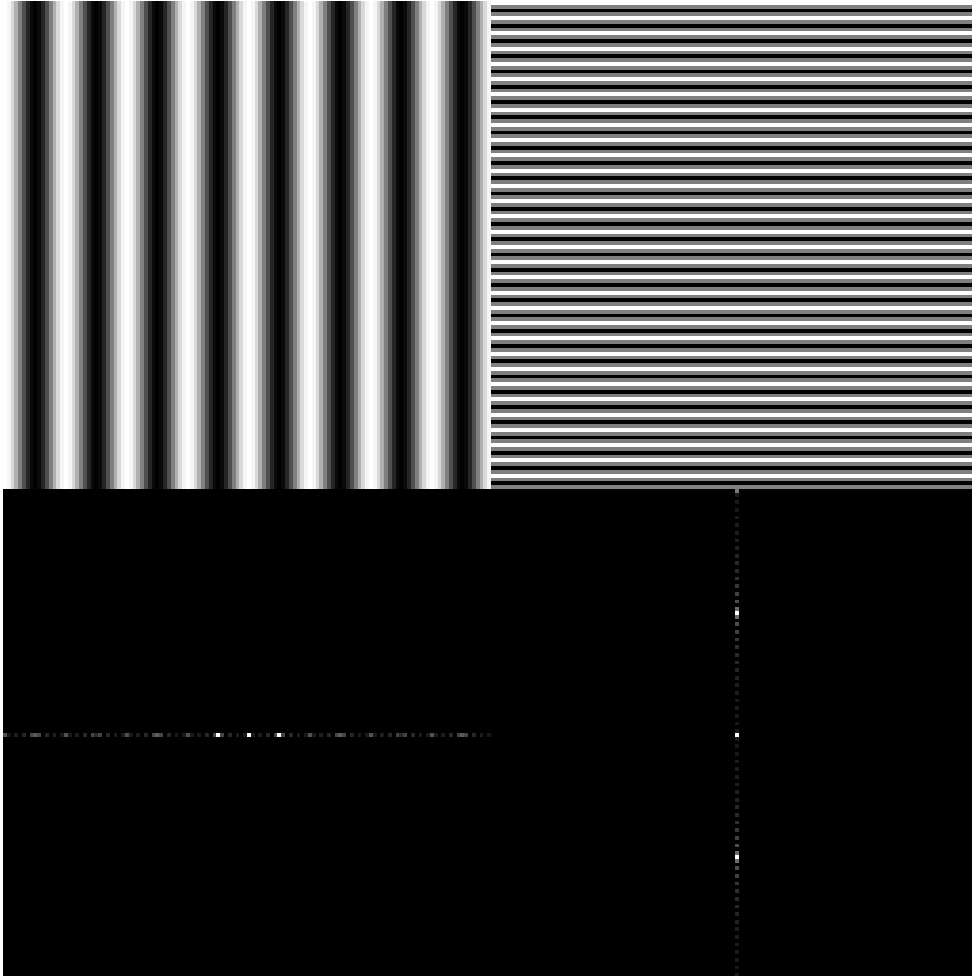
# Fourier Transform – *Interpreting the freq. domain*



# Fourier Transform – *Interpreting the freq. domain*

horizontal cos (low-f.)

vertical cos (high-f.)

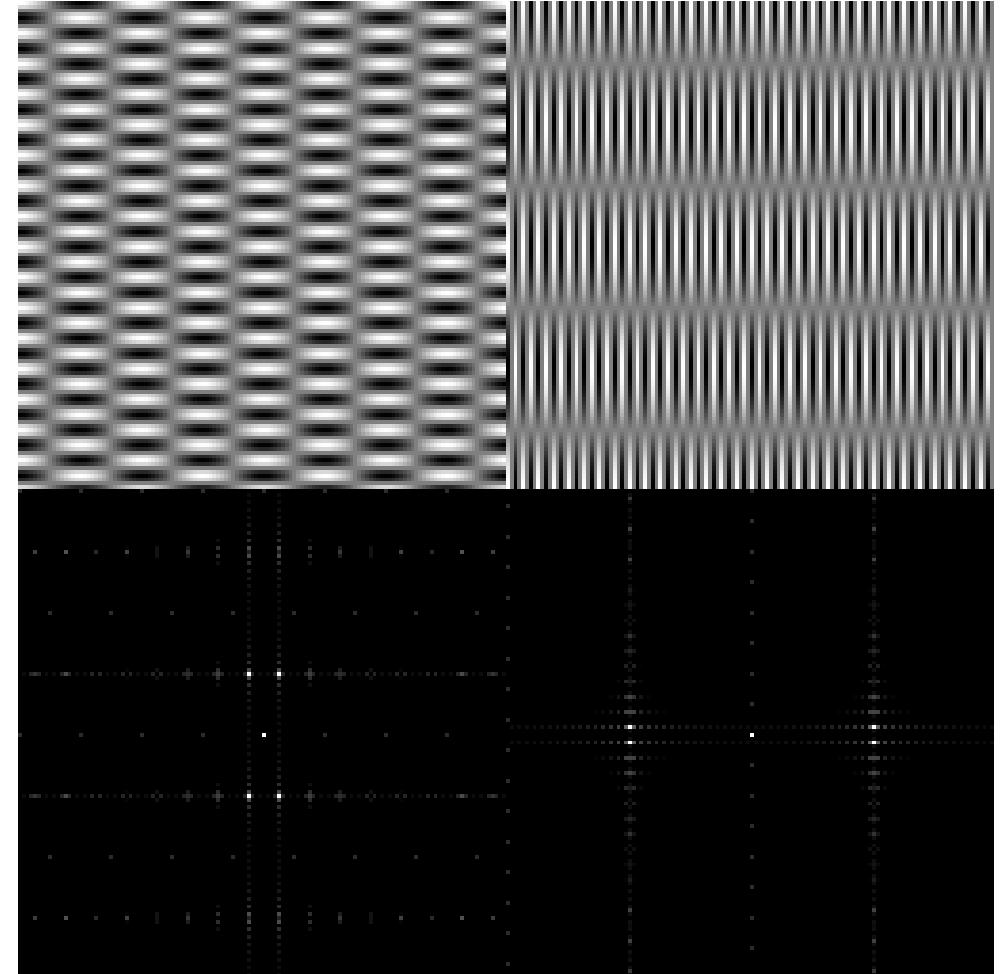
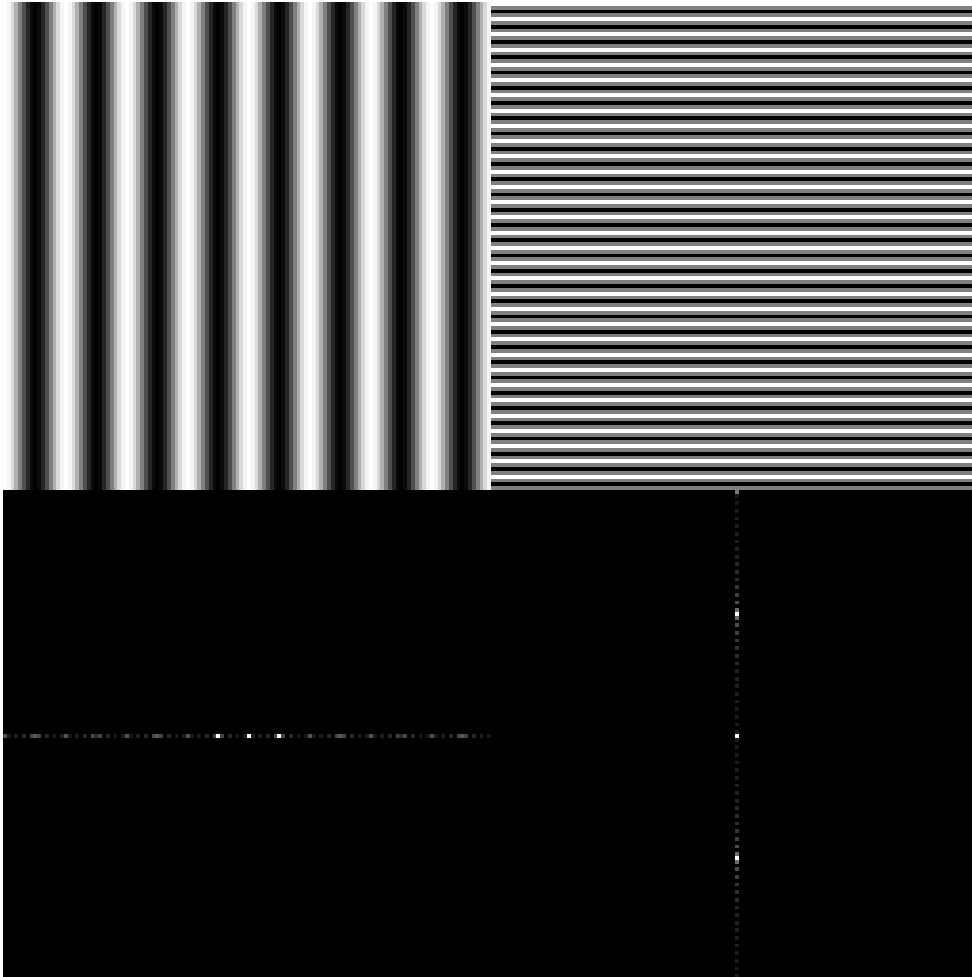


Fourier transform (**amplitude**)

# Fourier Transform – *Interpreting the freq. domain*

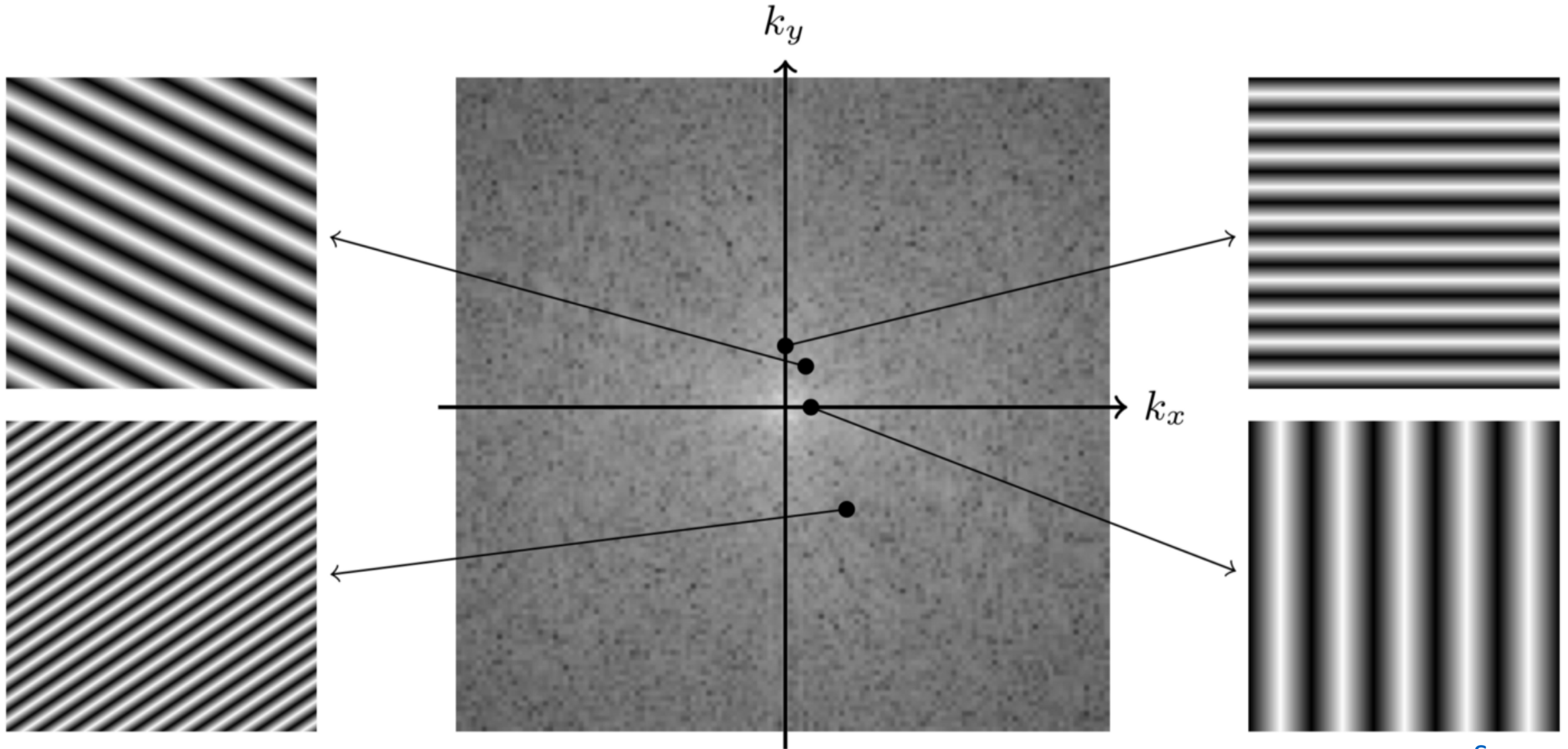
horizontal cos (low-f.)

vertical cos (high-f.)



Fourier transform (**amplitude**)

# Fourier Transform – *Interpreting the freq. domain*



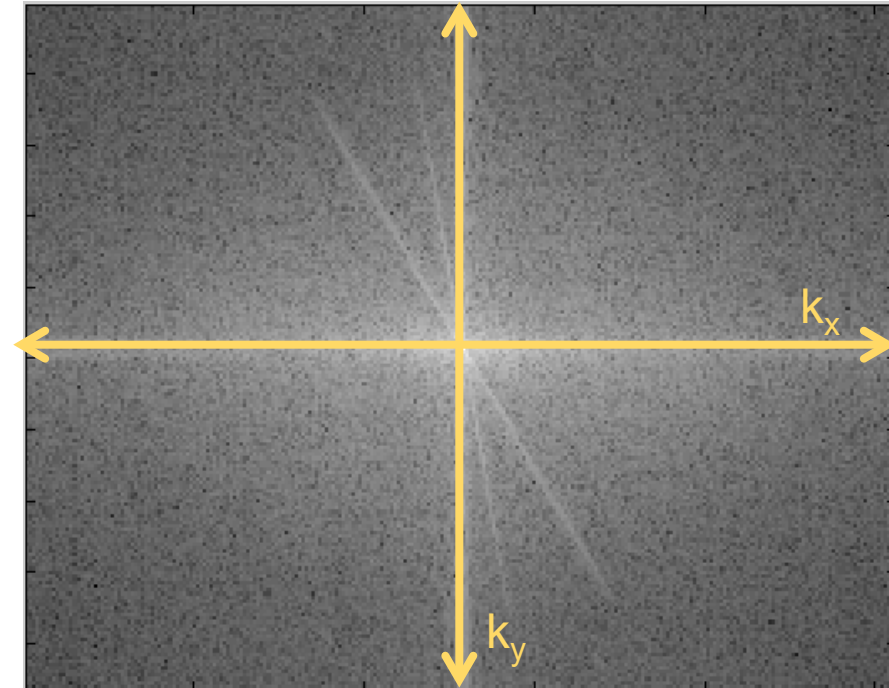
[Source](#)

# Fourier Transform – *Interpreting the freq. domain*

Natural image



Fourier decomposition  
Frequency coefficients (amplitude)

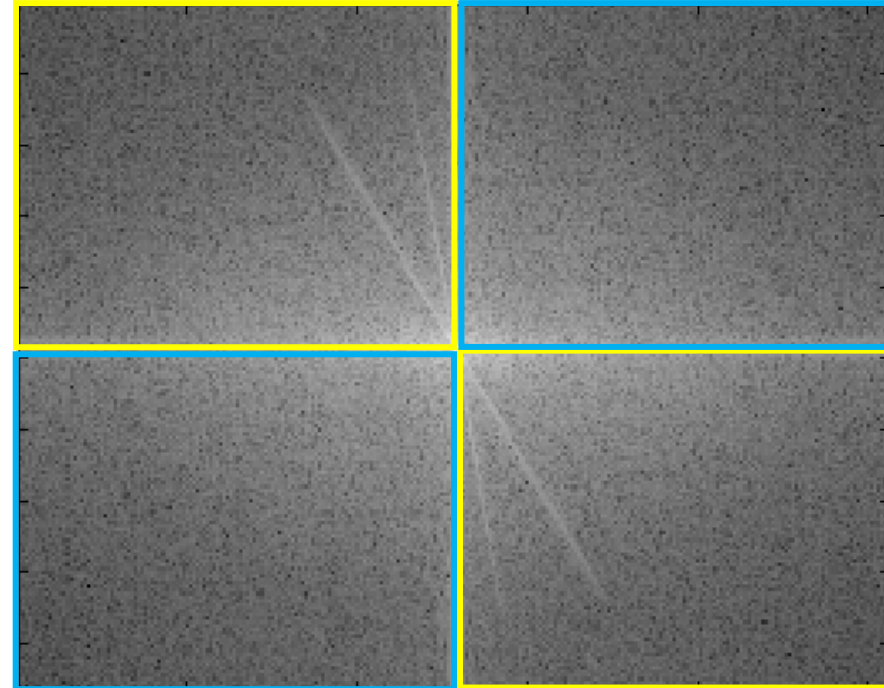


# Fourier Transform – *Interpreting the freq. domain*

Natural image



Fourier decomposition  
Frequency coefficients (amplitude)



# Fourier Transform – *Edge effects*

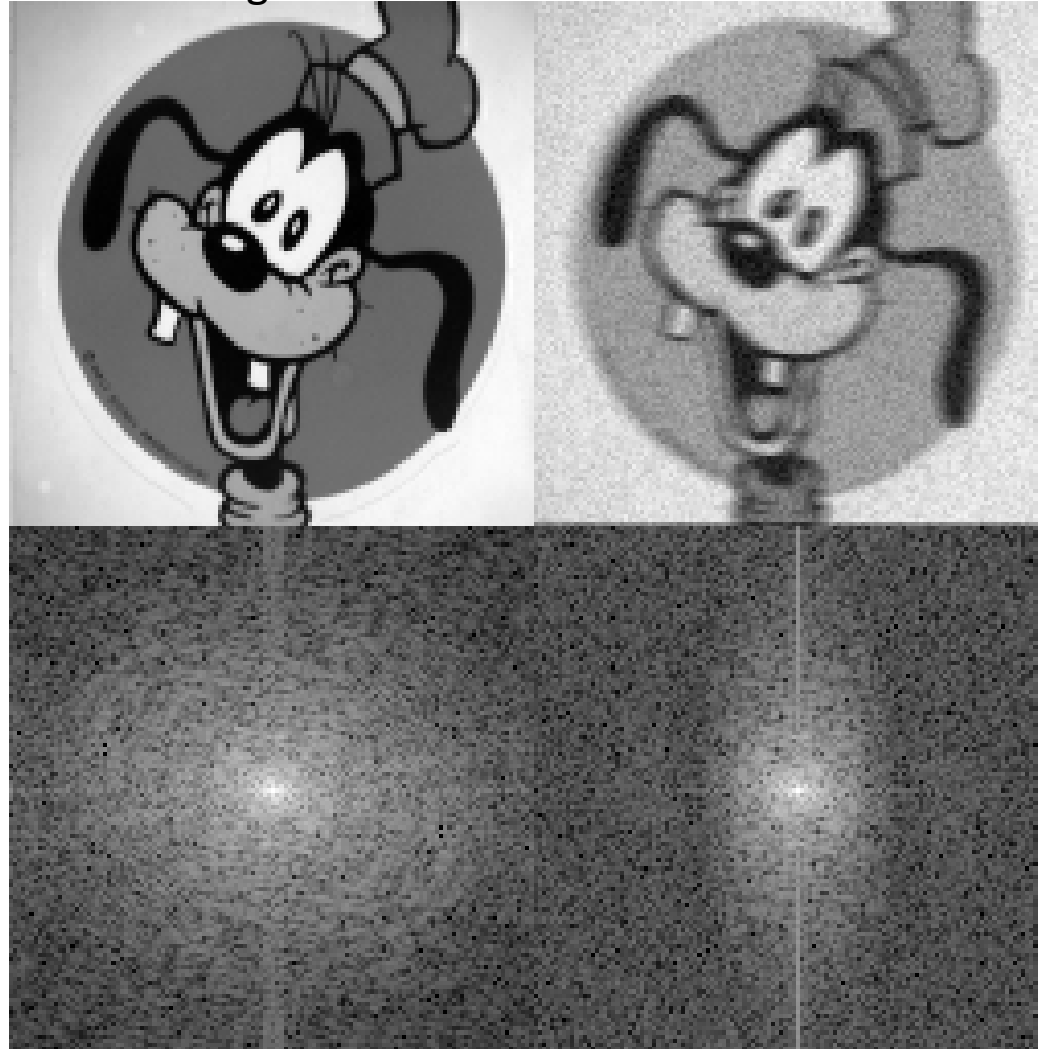


[Source](#)

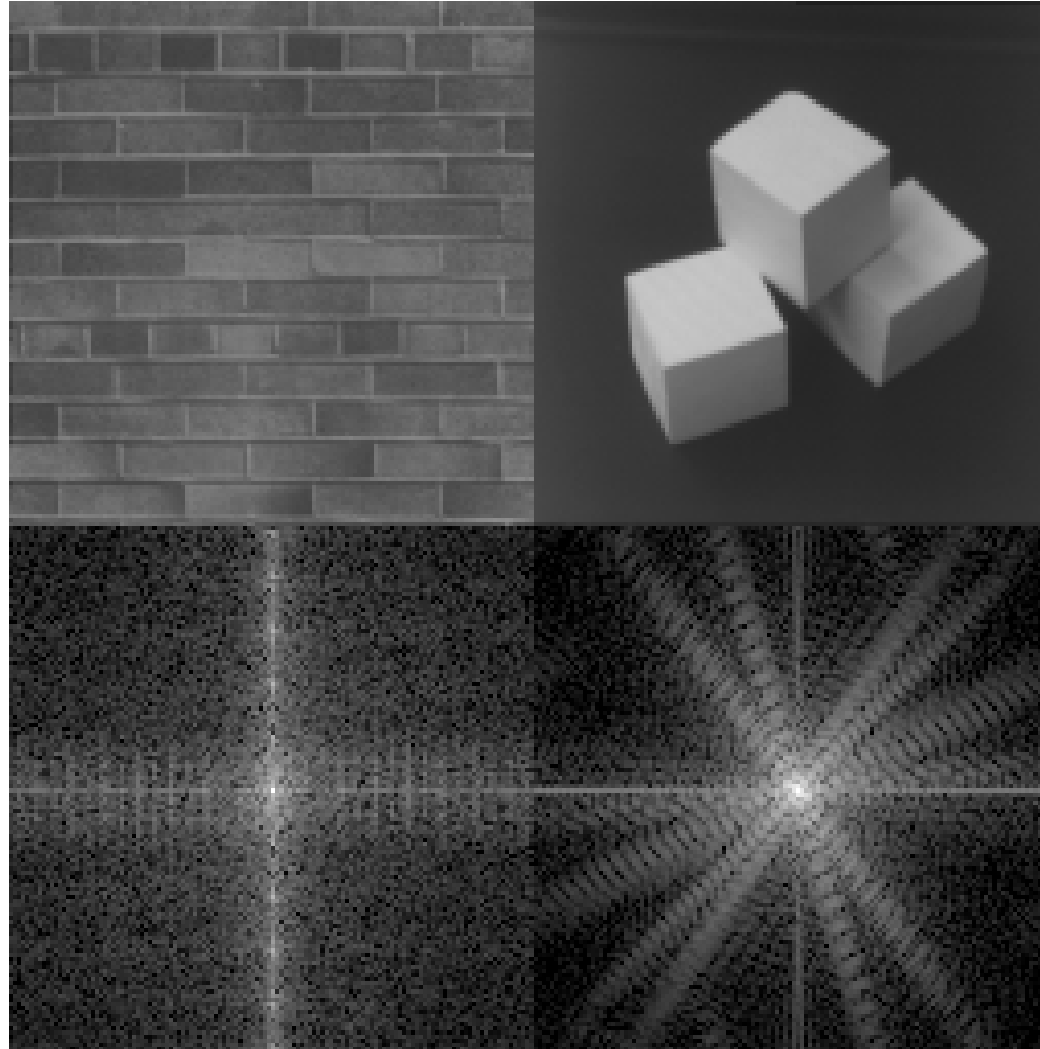
# Fourier Transform – *More examples*

Original

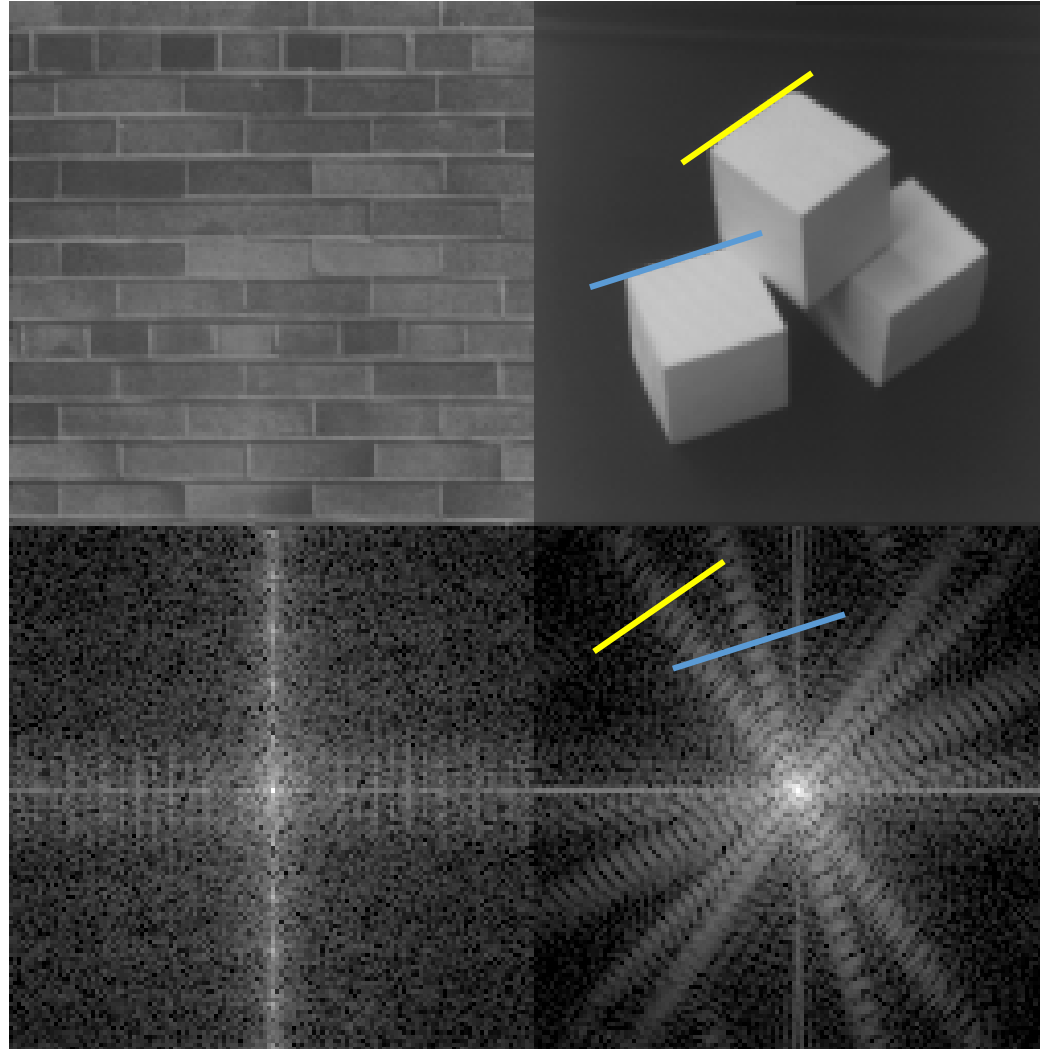
Smoothing horizontal dir.



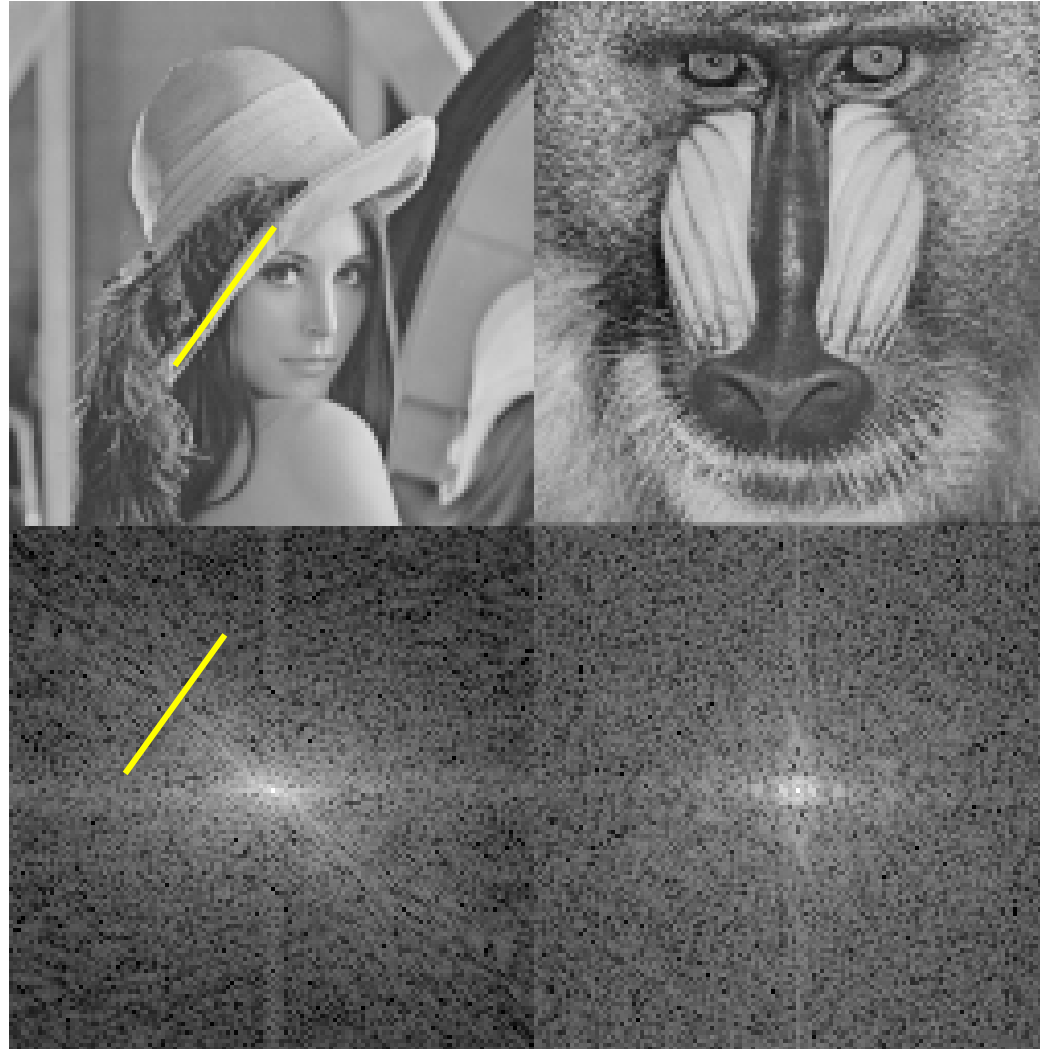
# Fourier Transform – *More examples*



# Fourier Transform – *More examples*



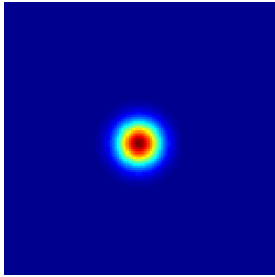
# Fourier Transform – *More examples*



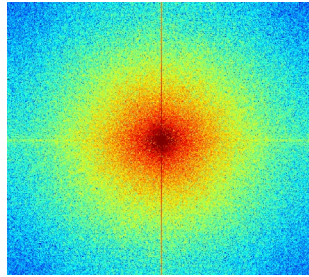
# Fourier Transform – *A pair-matching game*

Match the spatial domain image to the Fourier magnitude image

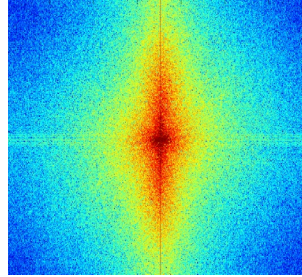
1



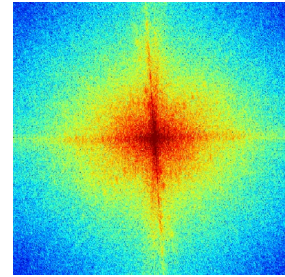
2



3



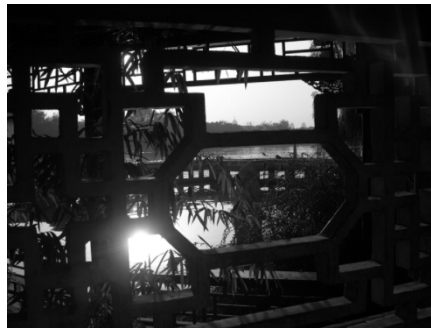
4



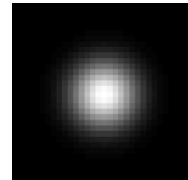
A



B



C

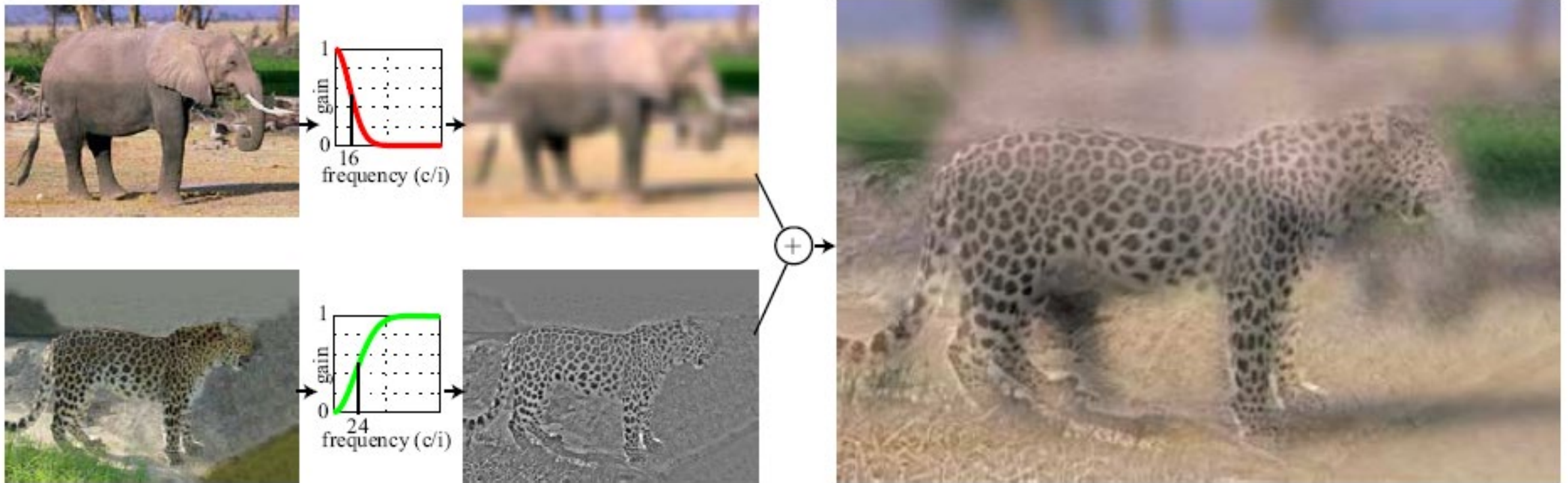


D



# Hybrid Images

Images that change in interpretation as a function of the viewing distance!



**High frequencies dominate perception** when available, but at a distance, only the low frequency (smooth) part of the signal can be seen.



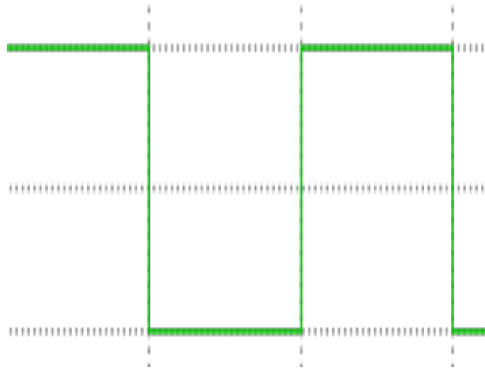
A. Oliva, A. Torralba, P.G. Schyns, [“Hybrid Images,”](#) SIGGRAPH 2006

# Questions?

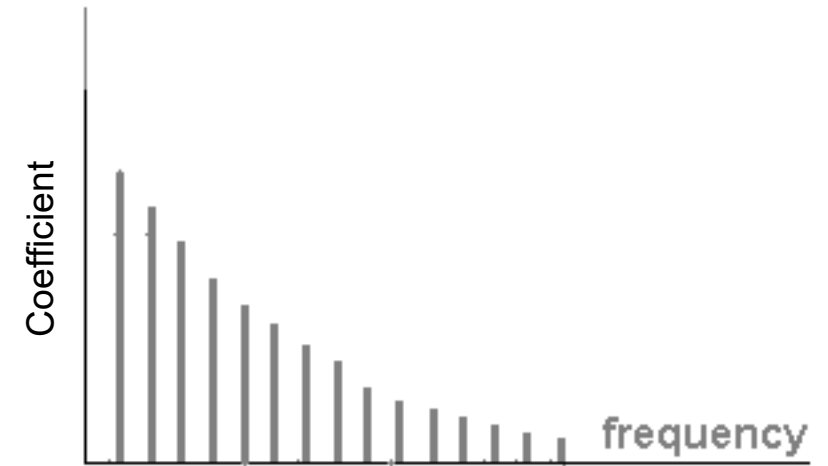
# So far...

- Images (actually, all signals) can be decomposed in frequencies.
- We can use Fourier series for that decomposition.
  - But how?
  - And can we get our image from these frequencies?

# Fourier Transform – *How do we reconstruct the signal?*



$$= A \sum_{f=1}^{\infty} \frac{1}{f} \sin(2\pi f t)$$

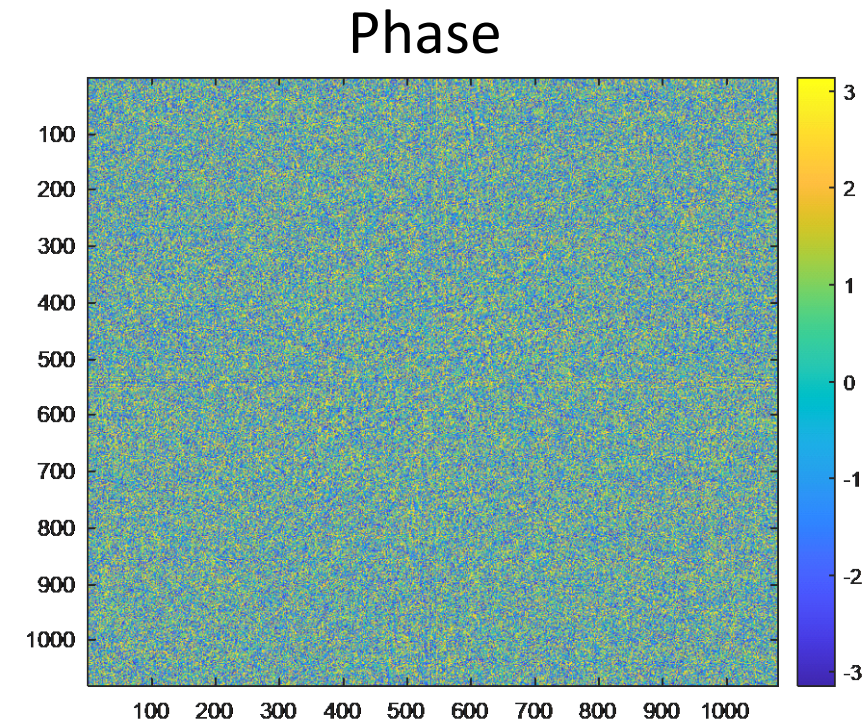
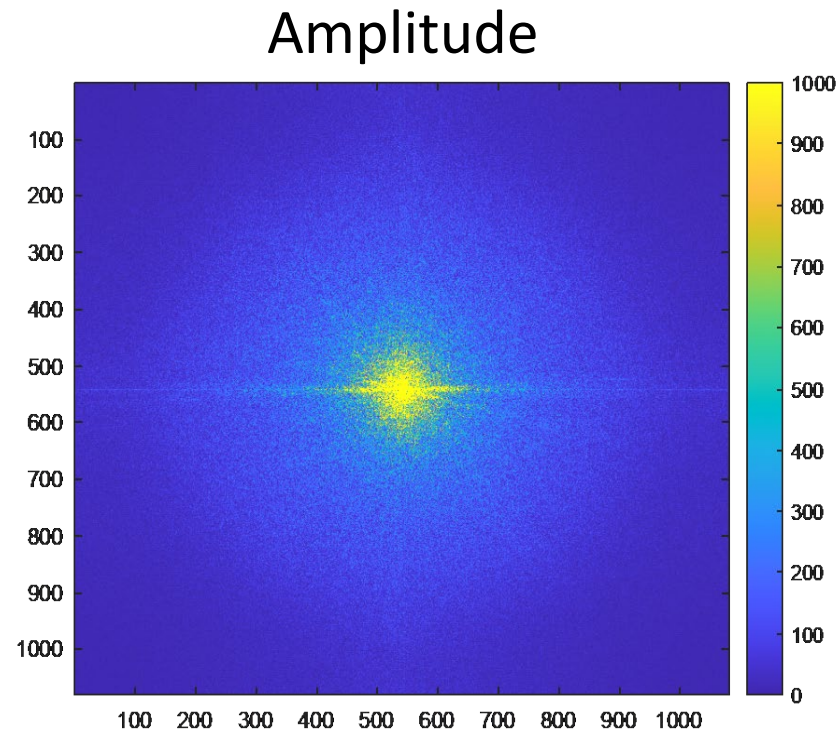


We just need to add to add the sinusoids multiplied by their amplitude.  
Right?

# Fourier Transform – *How do we reconstruct the signal?*



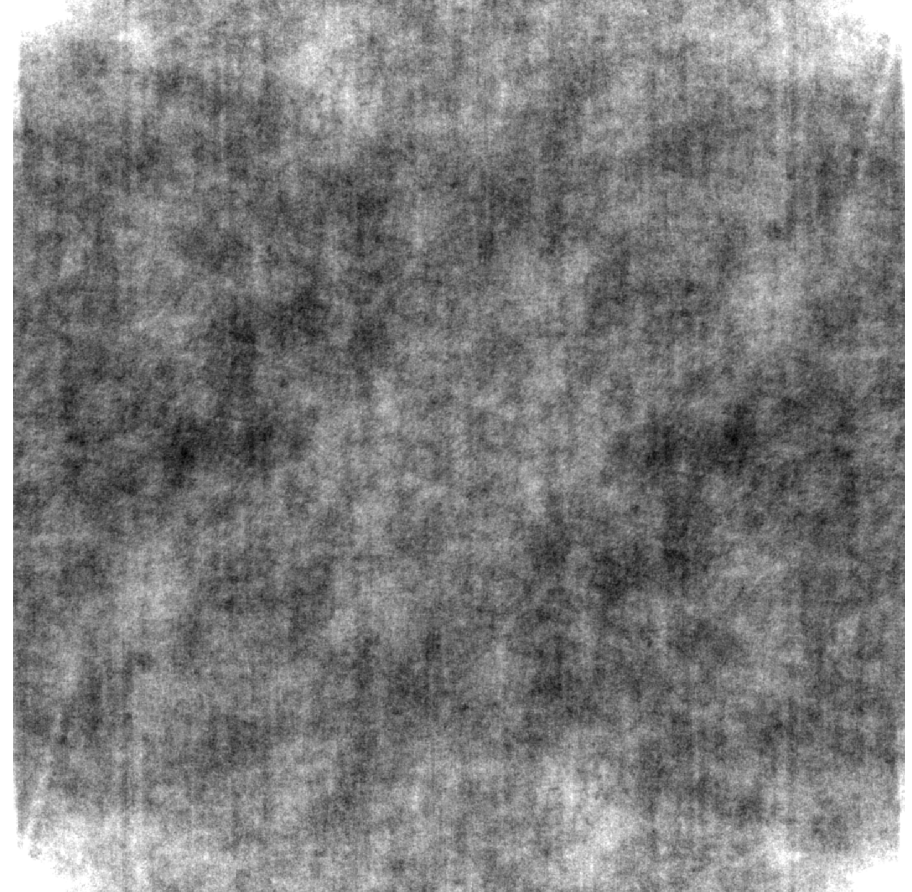
A wave is defined by its amplitude and frequency,  
**but also its PHASE**



# Fourier Series – *How do we reconstruct the signal?*

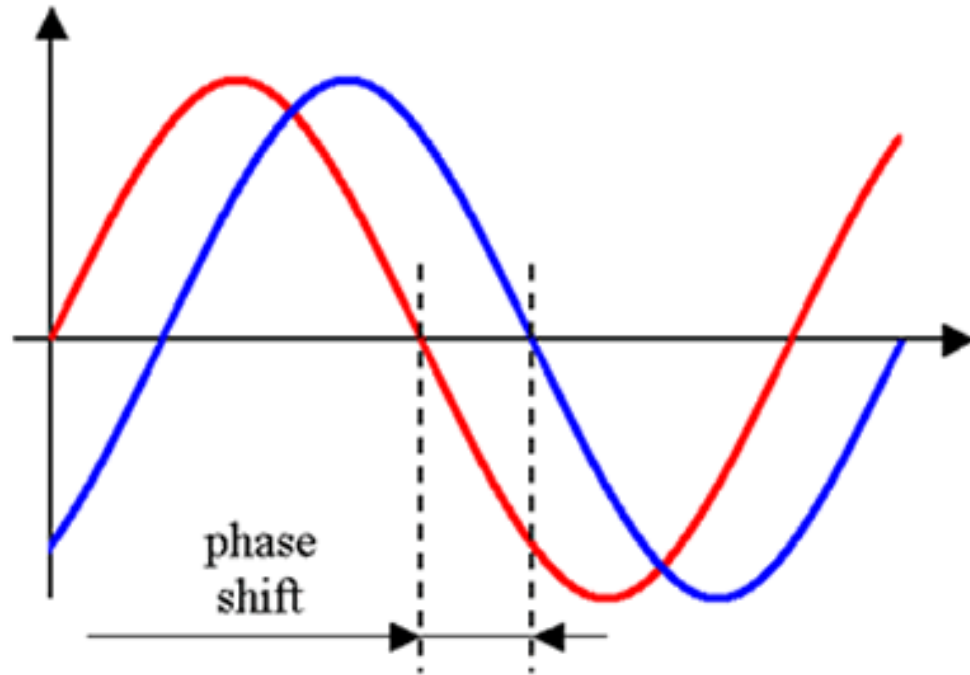


Original Image



Reconstruction from only the  
frequencies' amplitude

# Frequency Decomposition – *Amplitude & Phase*



- Amplitude tells you “how much”
- Phase tells you “where”

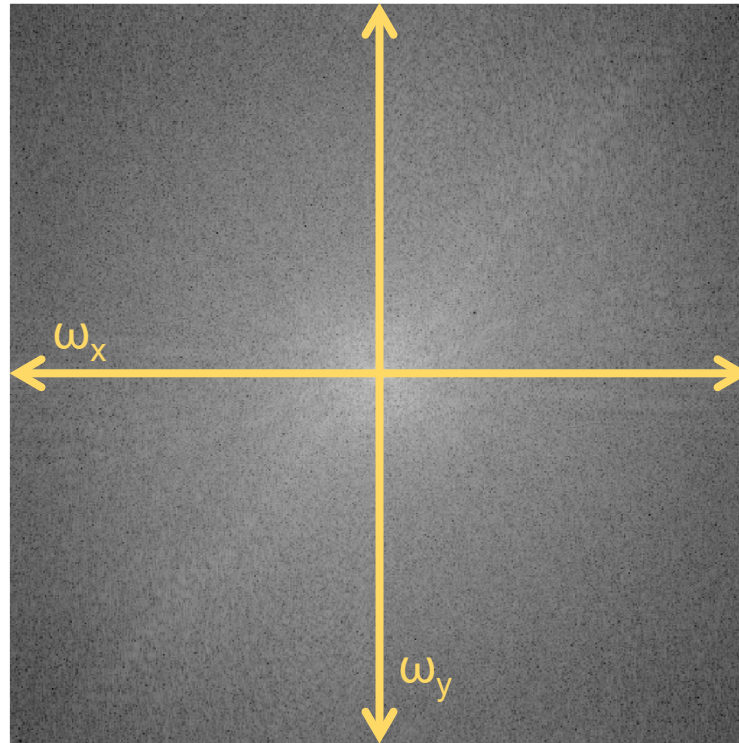
# Fourier Transform – *An example*

Fourier Transform  $\mathcal{F}(I)$

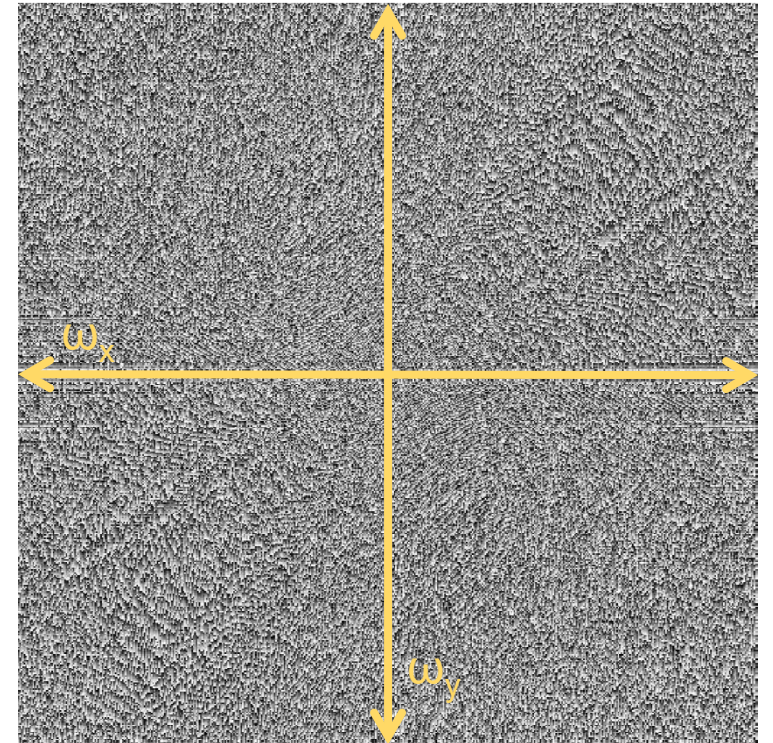
Image  $I$



Amplitude



Phase



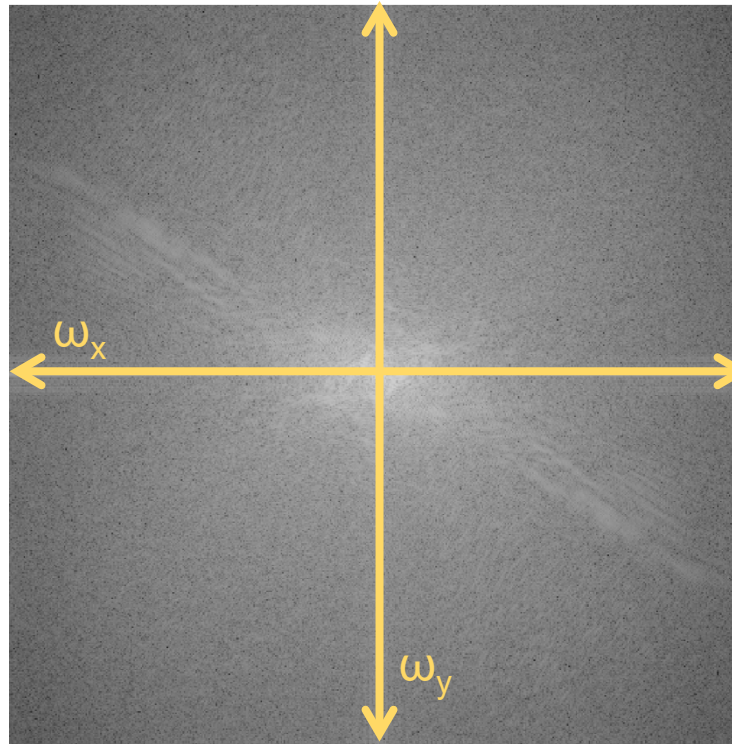
# Fourier Transform – *An example*

Fourier Transform  $\mathcal{F}(I)$

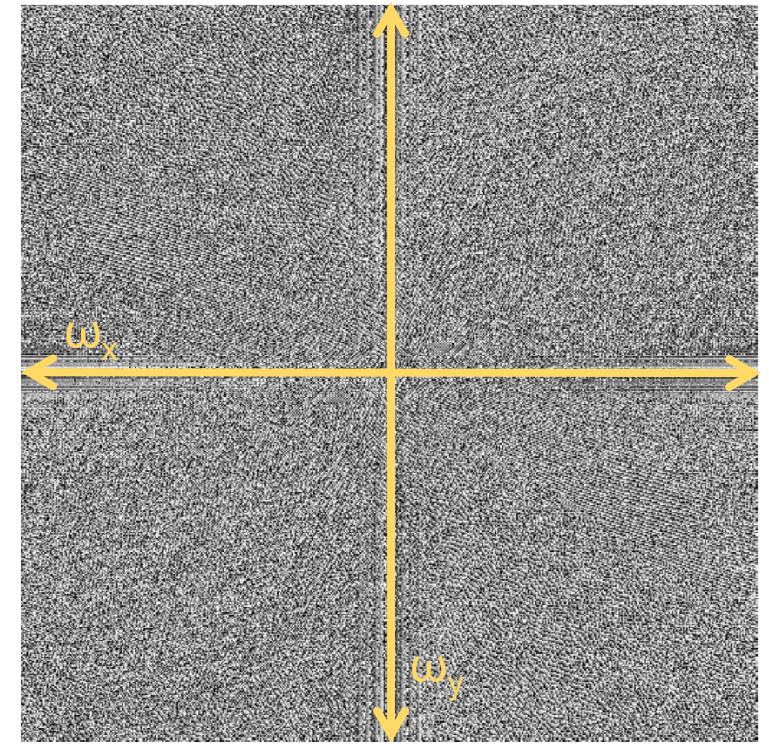
Image  $I$



Amplitude

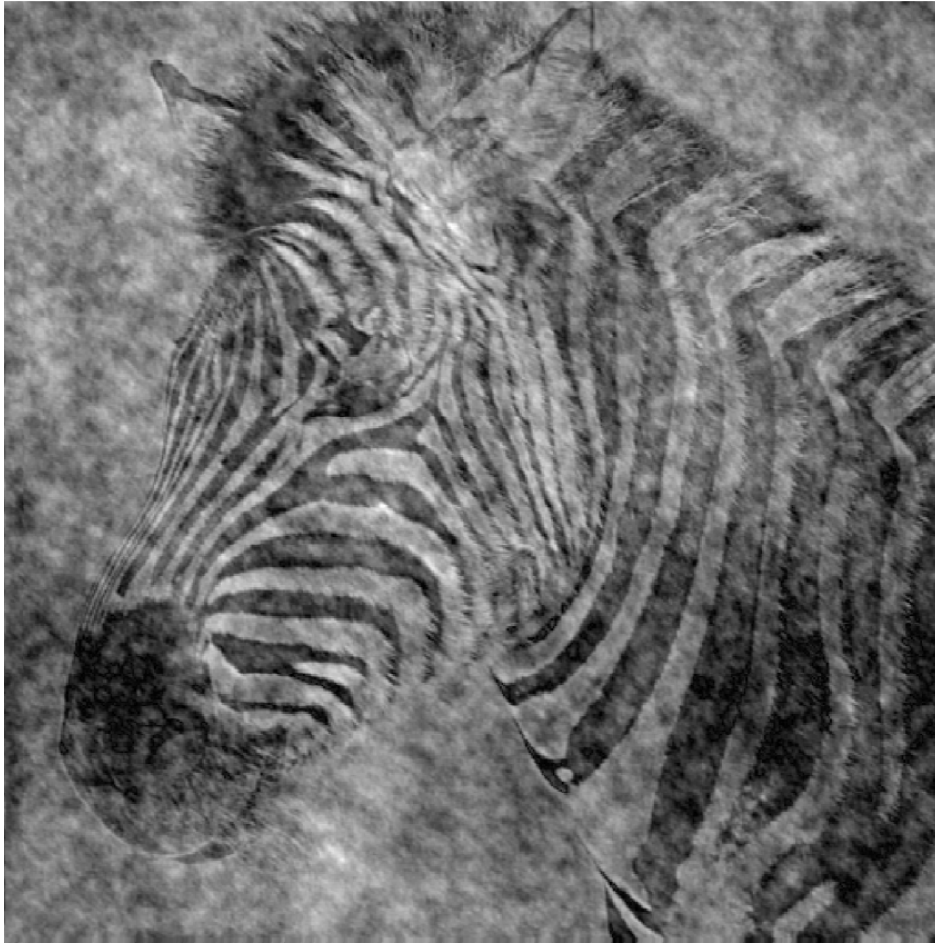


Phase

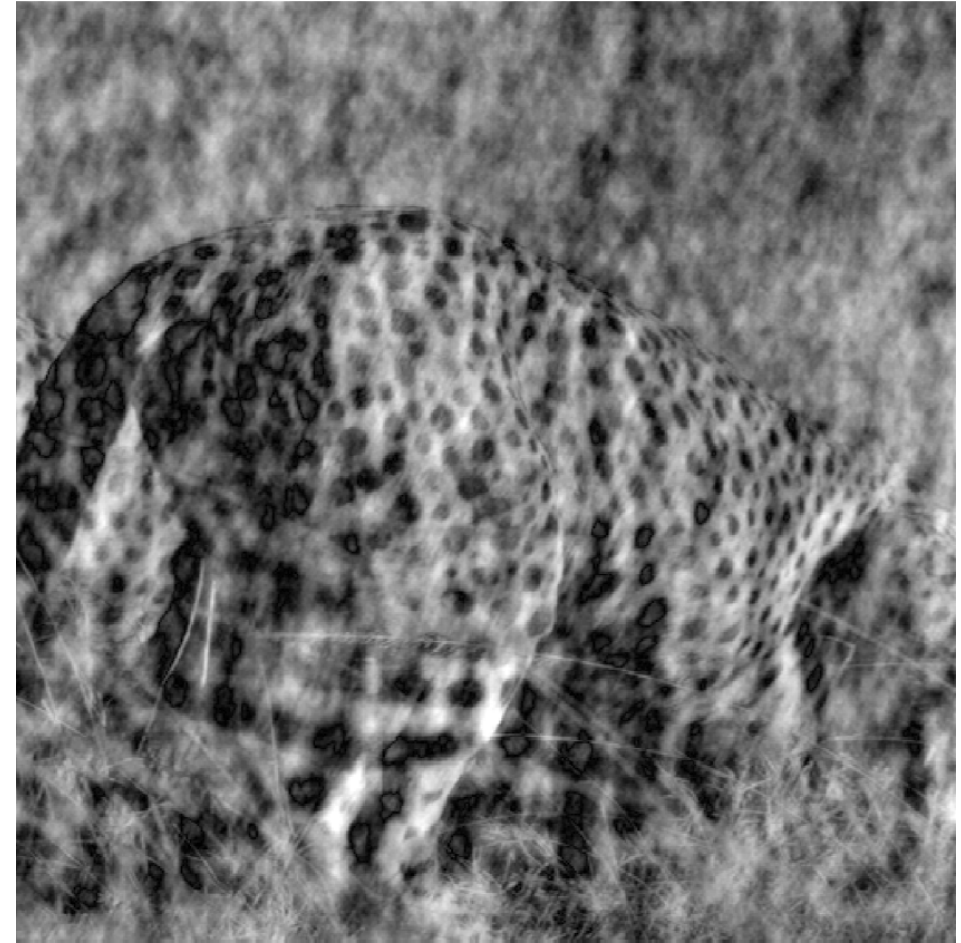


# Fourier Transform – *Behold the Cheedra!*

Zebra phase, cheetah amplitude



Cheetah phase, zebra amplitude



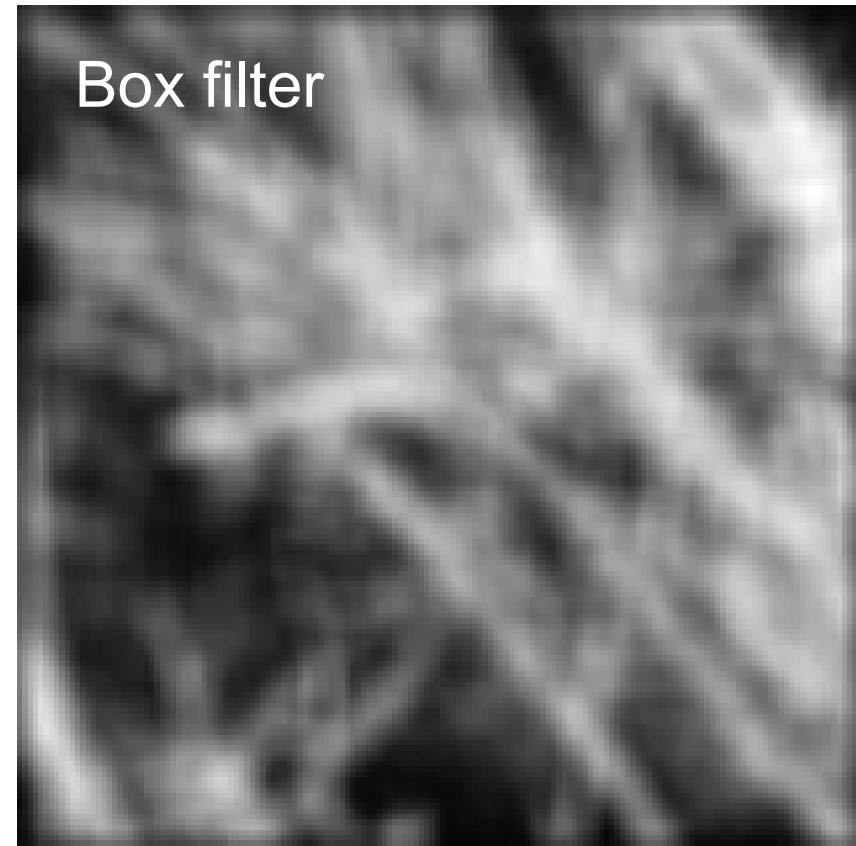
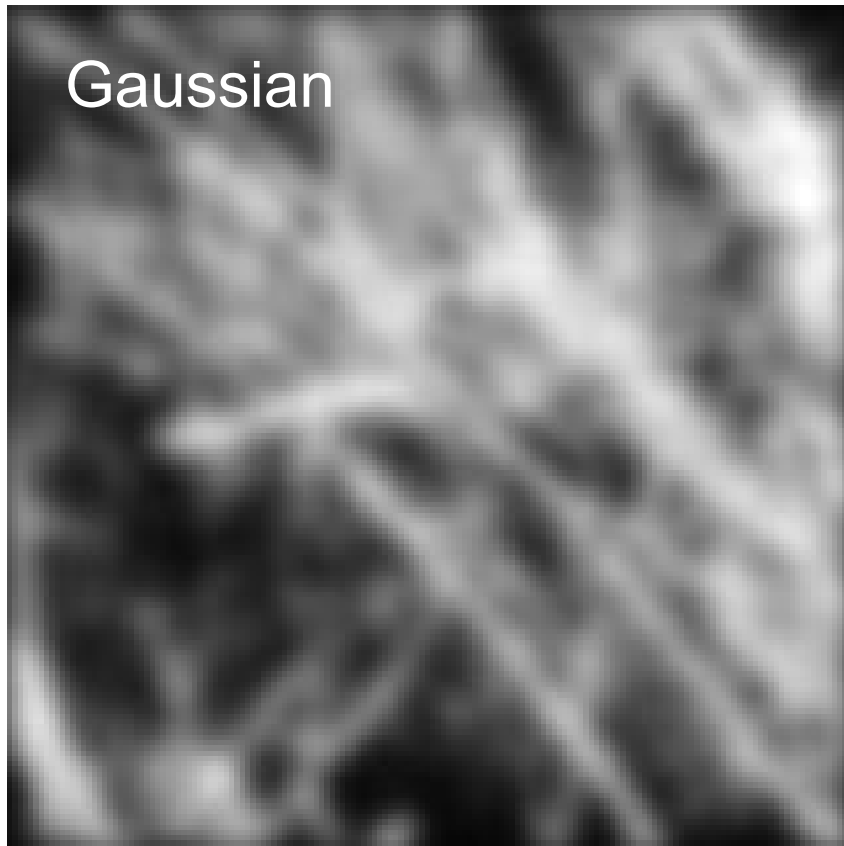
# Fourier Transform – *Test*

- In the Fourier domain: Where is more of the information that we see in the visual world?
  - a) Frequency Amplitude
  - b) Frequency Phase
- Most information in the image is carried in the phase, not the amplitude
  - The magnitude of the FFT is *how much energy each of the waves carry*. The phase is *how those waves are positioned (location of the features)*.
- The frequency amplitude of natural images are quite similar
  - Heavy in low frequencies, falling off in high frequencies

# Fourier Transform – *Analyzing filters*

Why does the **Gaussian** filter give a nice smooth image?

Why the **box filter** doesn't (edgy artifacts)?

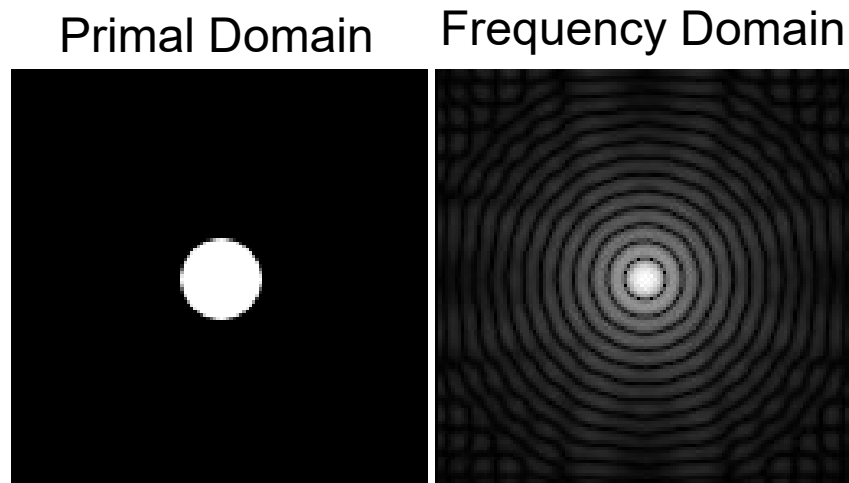


# Fourier Transform – *Analyzing filters*

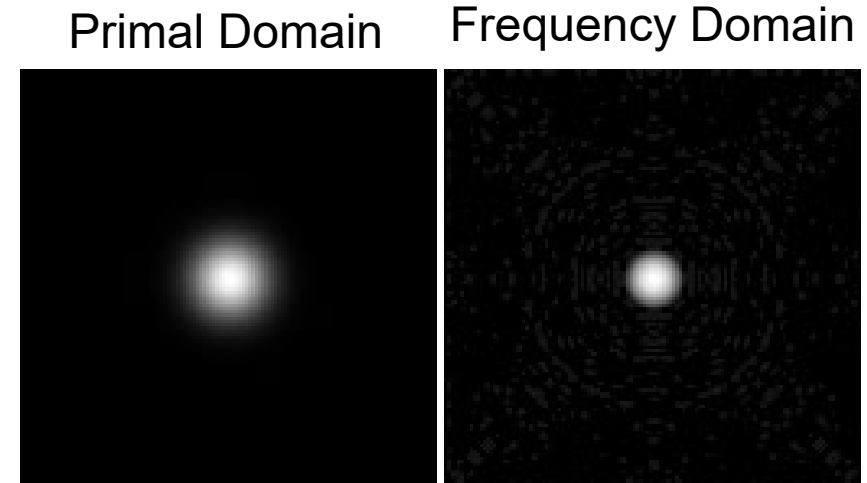
Why does the **Gaussian** filter give a nice smooth image?

Why the **box filter** doesn't (edgy artifacts)?

>> *"The answer, my friend, is blowing in the Fourier domain"*



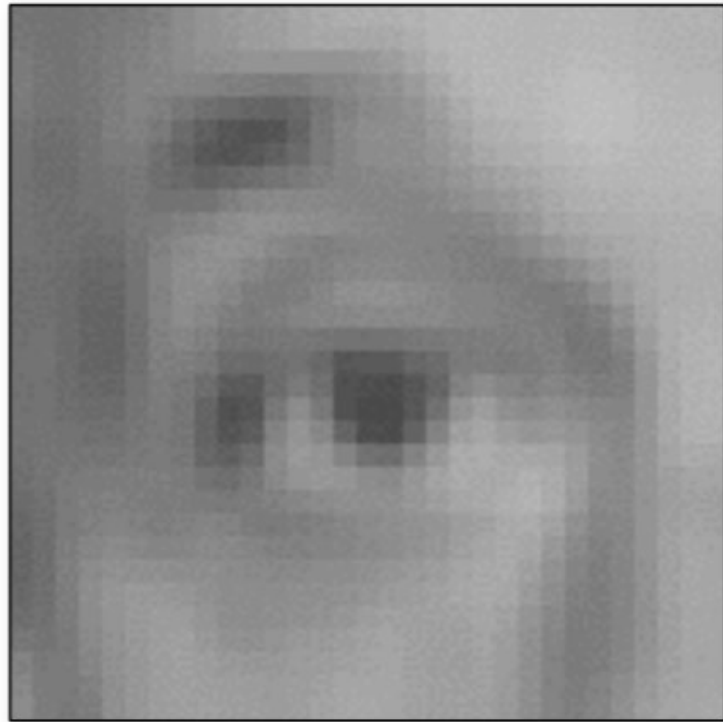
Box Filter



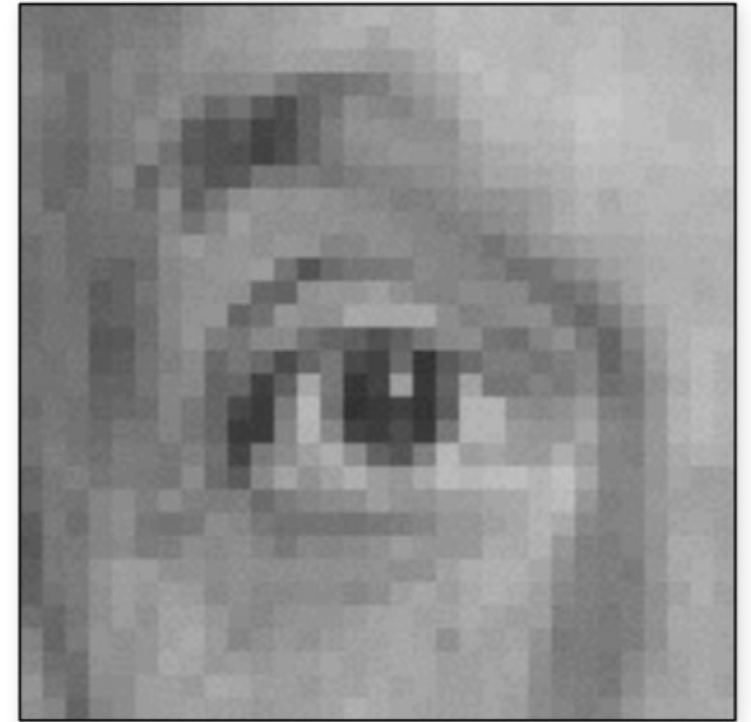
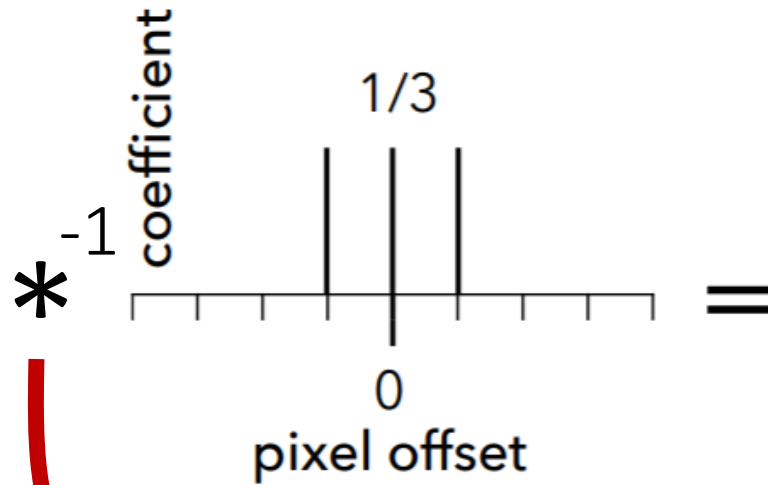
Gaussian Filter

# Questions?

# ... and can we go the other way around?



blurred  
(applied in both dimensions)



original

*Far from trivial*

# Fourier Transform – *The Convolution Theorem*

- **Corollary:** The Fourier transform of the **deconvolution** of two functions is the division of their Fourier transforms

$$F[g *^{-1} h] = F[g] / F[h]$$

Is this true?

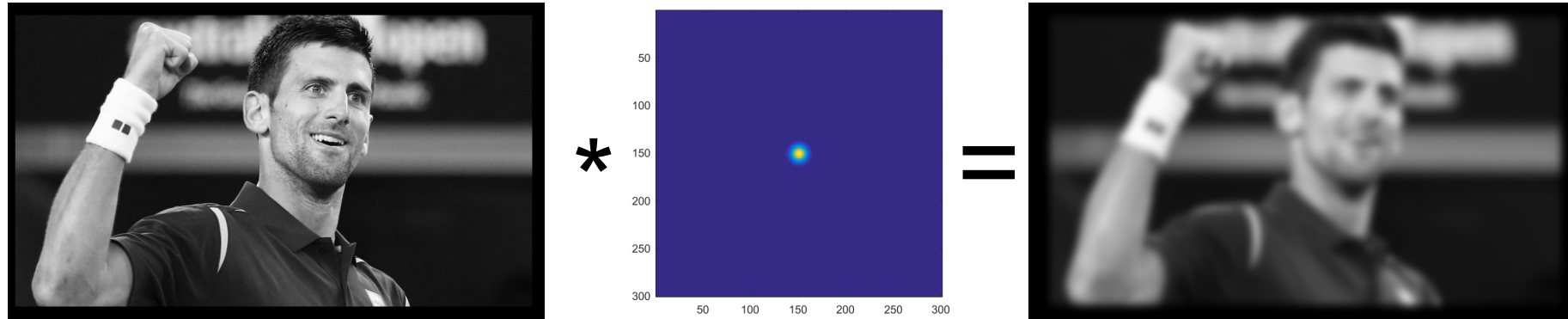
- **Deconvolution** division in frequency domain!

$$g *^{-1} h = F^{-1} [F[g] / F[h]]$$

# Fourier Transform – *Is convolution invertible?*

- If convolution is just multiplication in the Fourier domain...  
... isn't deconvolution just division?
- Sometimes, it clearly is invertible (e.g. a convolution with an identity filter)
- In one case, it clearly isn't invertible (e.g. convolution with an all zero filter)
- What about for common filters like a Gaussian?

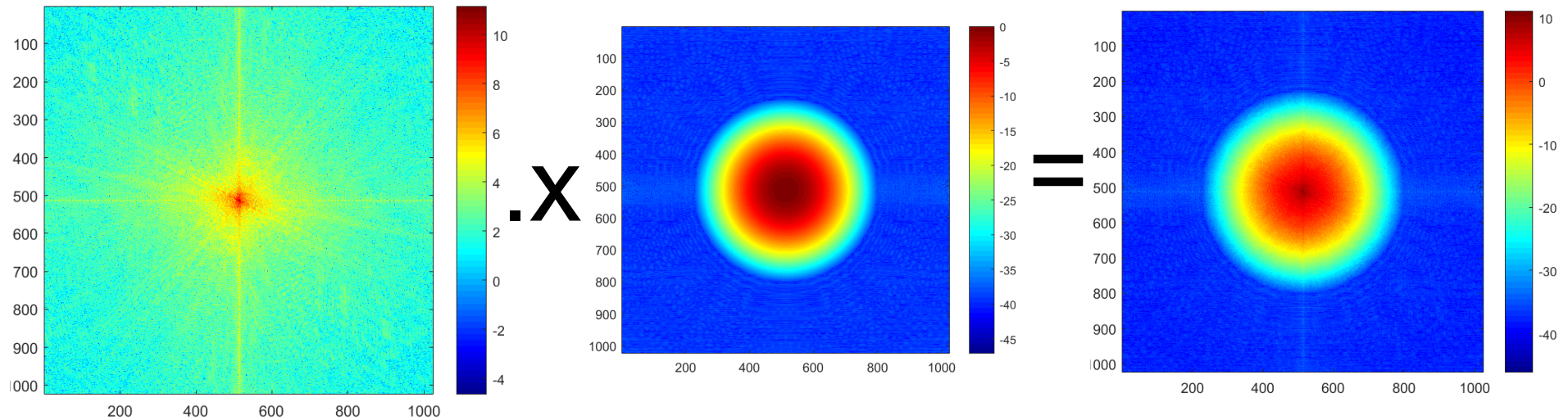
# Fourier Transform – *Convolution*



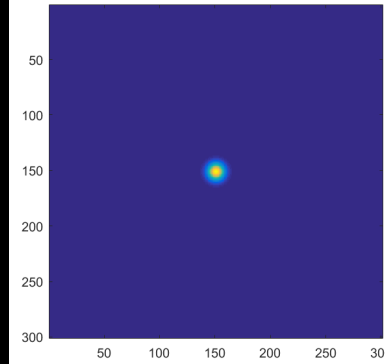
FFT ↓

FFT ↓

iFFT ↑



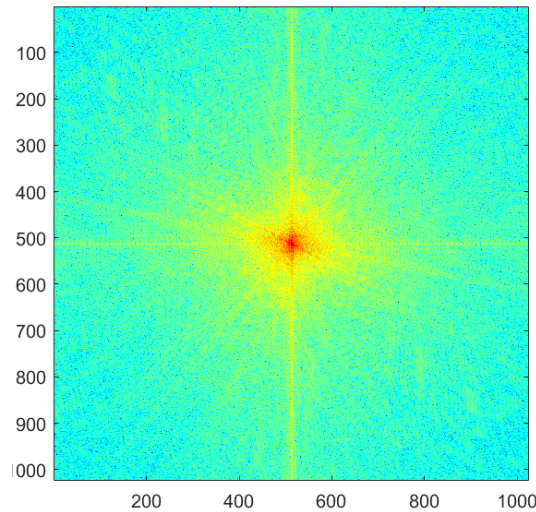
# Fourier Transform – *Deconvolution*?



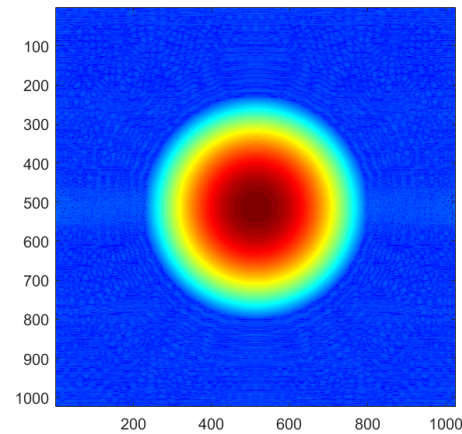
iFFT ↑

FFT ↓

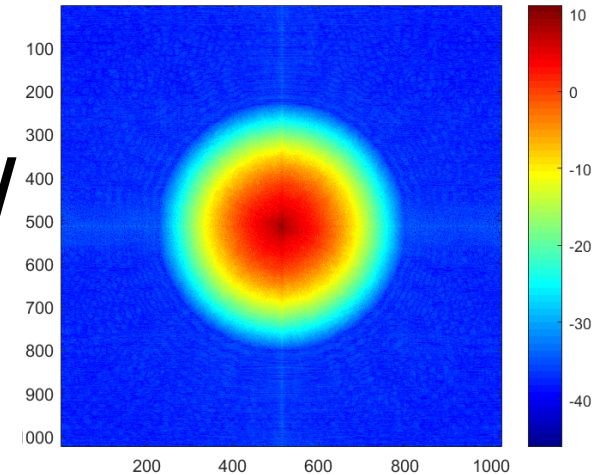
FFT ↓



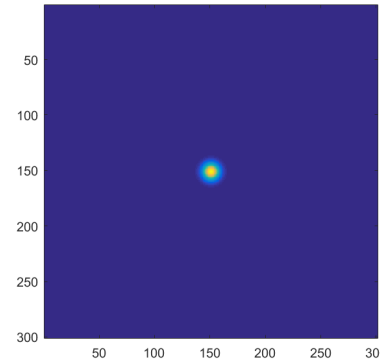
=



./



# But under more realistic conditions...



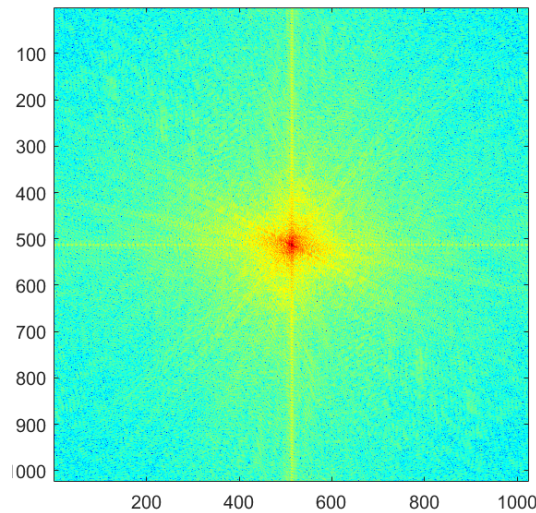
Random noise, .000001 magnitude



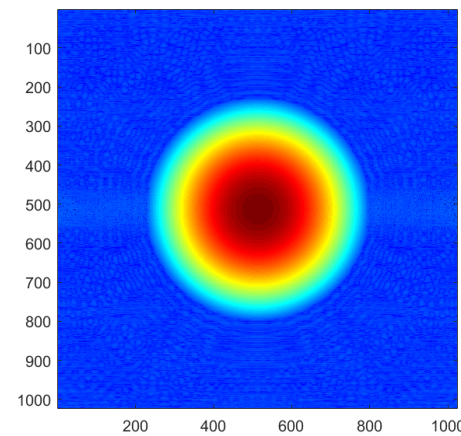
iFFT ↑

FFT ↓

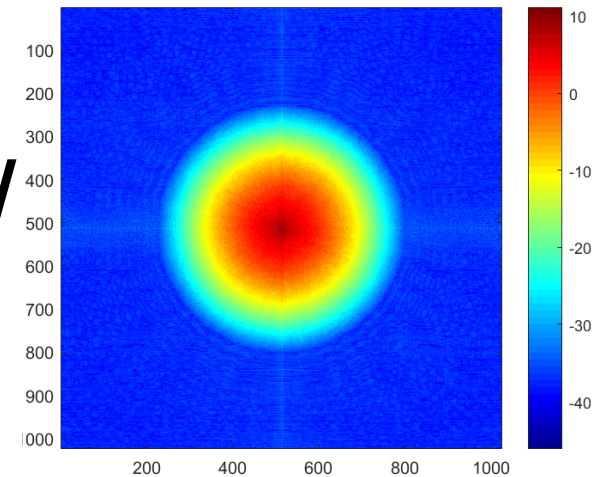
FFT ↓



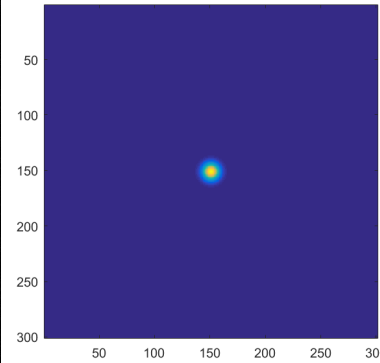
=



./



# But under more realistic conditions...



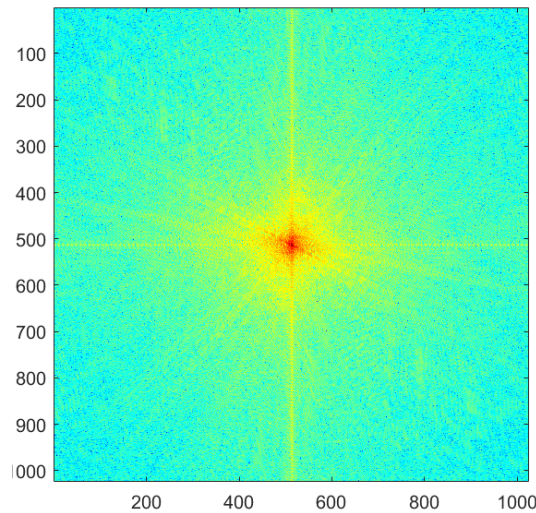
Random noise, .0001 magnitude



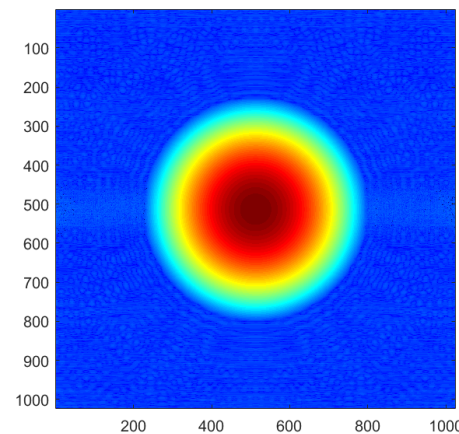
iFFT ↑

FFT ↓

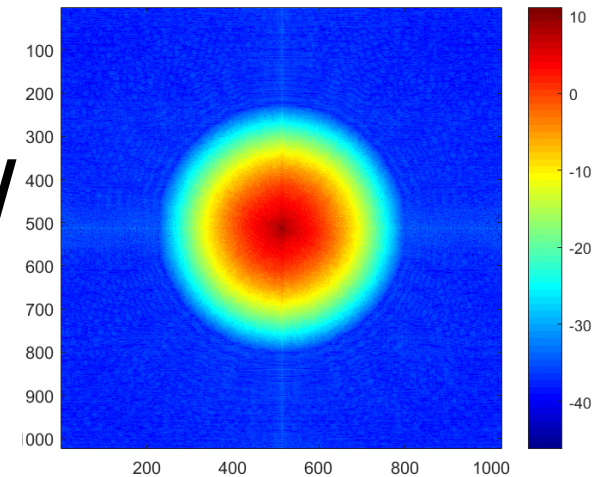
FFT ↓



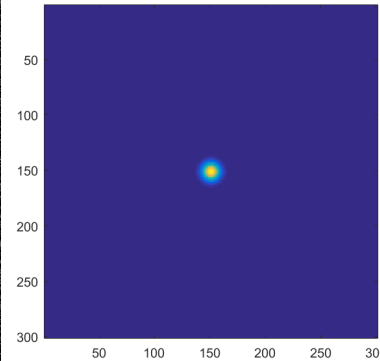
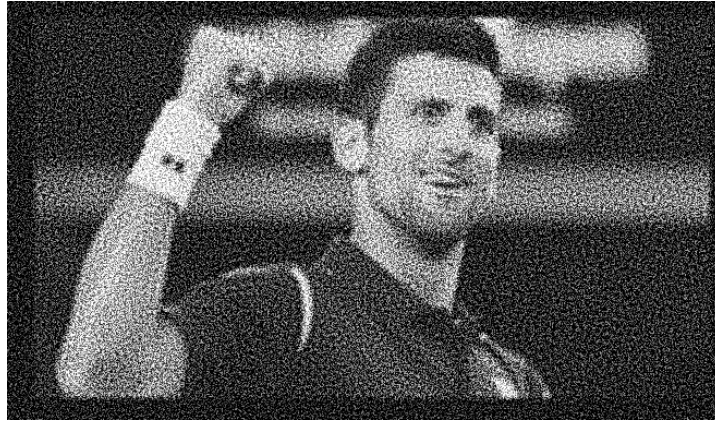
=



./



# But under more realistic conditions...



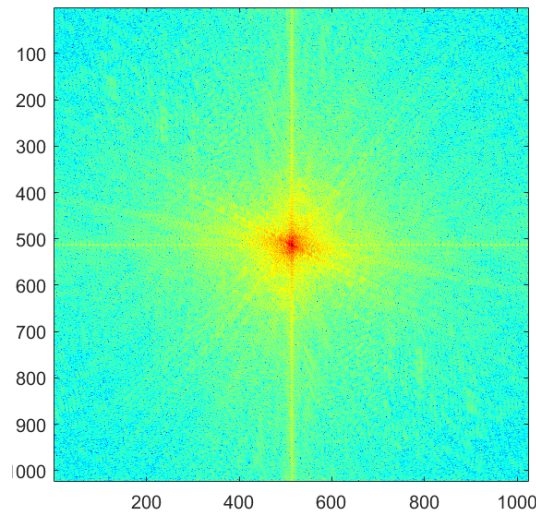
Random noise, .001 magnitude



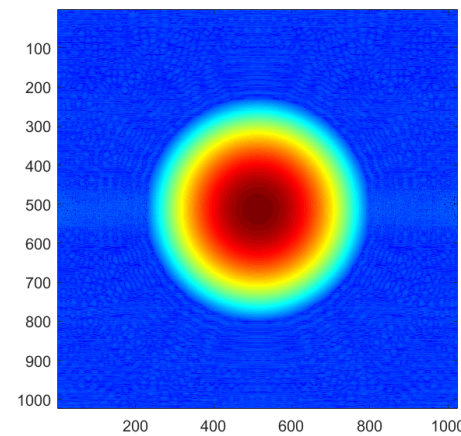
iFFT ↑

FFT ↓

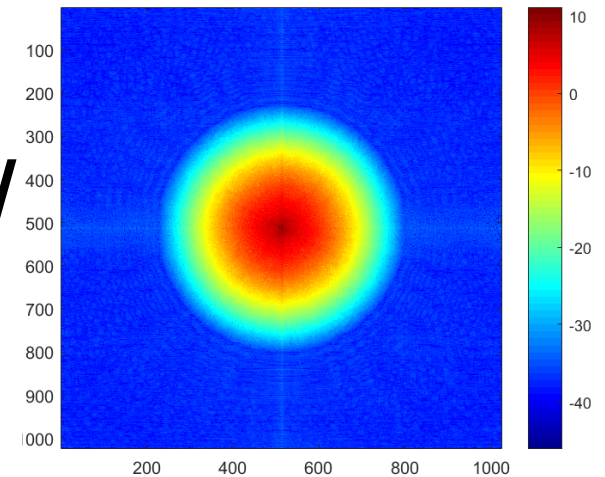
FFT ↓



=

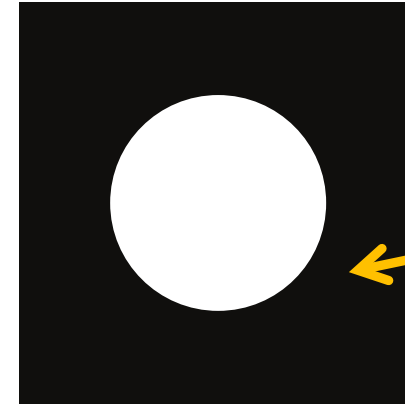


÷



# Fourier Transform – *Deconvolution*

- The OTF (Fourier of PSF) is a low-pass filter



zeros at high  
frequencies

- The measured signal includes noise

$$B = k * I + n$$



noise term

- When we divide by zero (or a very small number), we amplify the high frequency noise

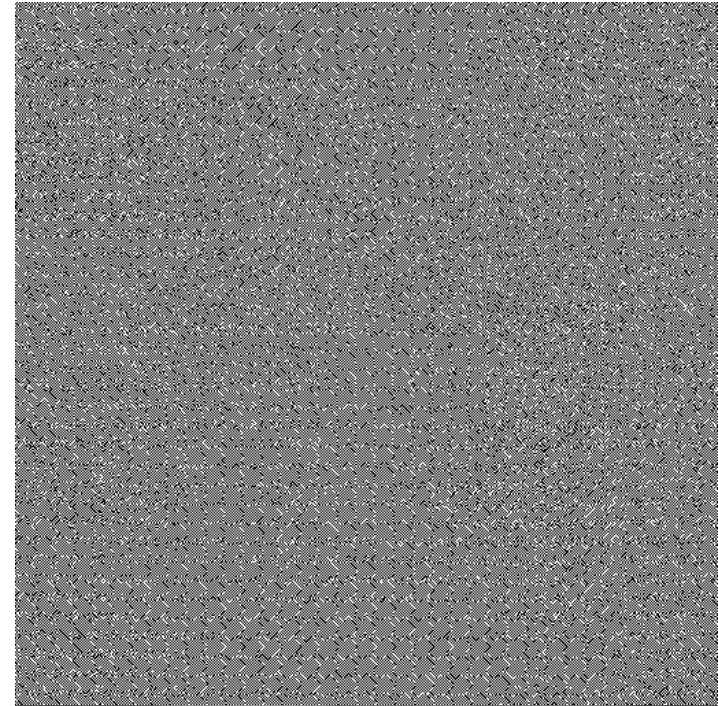
# Fourier Transform – *Naïve Deconvolution*

Even tiny noise can make the results awful.

- Example for Gaussian of  $\sigma = 0.05$



$$* \left[ \text{Gaussian Kernel} \right]^{-1} =$$



$$B * k^{-1} = I_{\text{est}}$$

# Fourier Transform – *Wiener Deconvolution*

Apply inverse kernel and do not divide by zero:

$$I_{\text{est}} = F^{-1} \left( \frac{|F(k)|^2}{|F(k)|^2 + 1/\text{SNR}(\omega)} \cdot \frac{F(B)}{F(k)} \right)$$

noise-dependent damping factor



- Derived as solution to maximum-likelihood problem under Gaussian noise assumption
- Requires noise of signal-to-noise ratio at each frequency

$$\text{SNR}(\omega) = \frac{\text{signal variance at } \omega}{\text{noise variance at } \omega}$$

# Fourier Transform – *Wiener Deconvolution*

Apply inverse kernel and do not divide by zero:

$$I_{\text{est}} = F^{-1} \left( \frac{|F(k)|^2}{|F(k)|^2 + 1/\text{SNR}(\omega)} \cdot \frac{F(B)}{F(k)} \right)$$

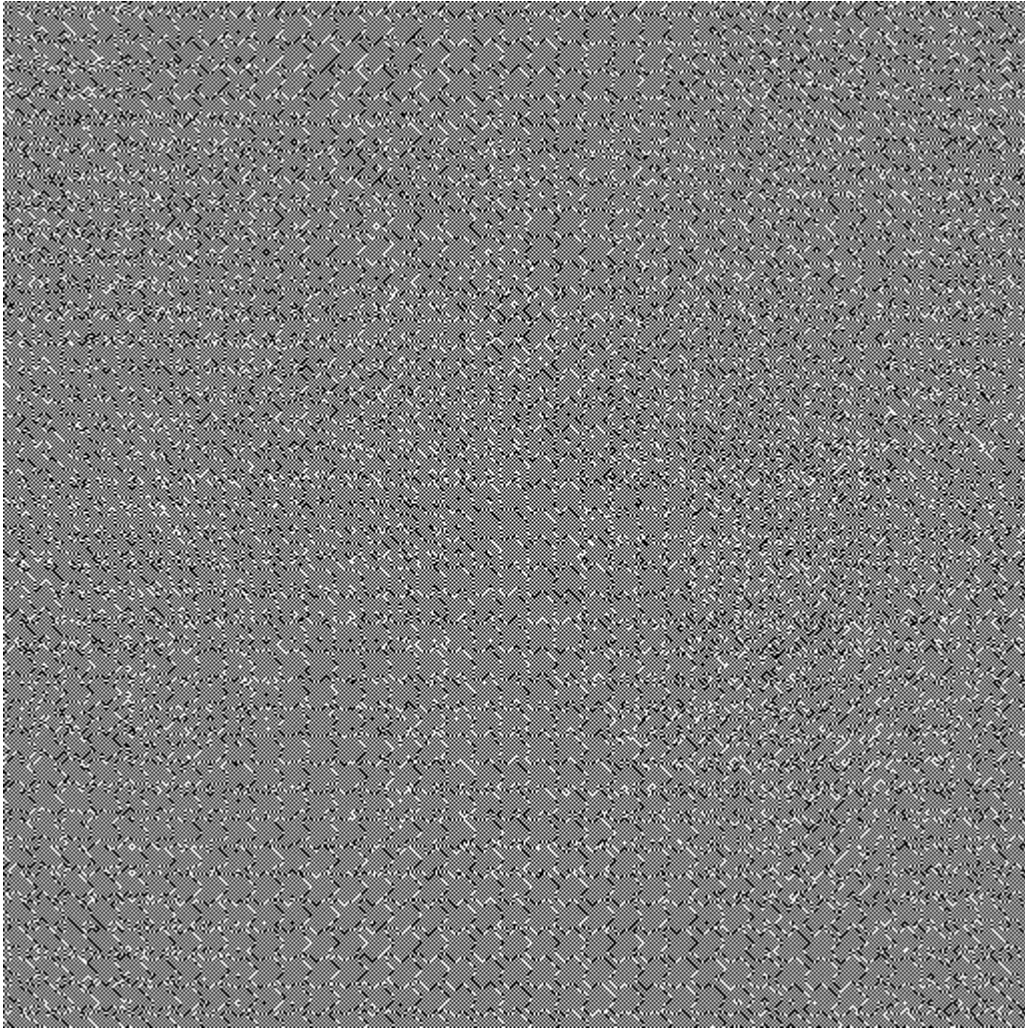
noise-dependent damping factor



Intuitively:

- When SNR is high (low or no noise), just divide by kernel.
- When SNR is low (high noise), just set to zero.

# Fourier Transform – *Deconvolution Comparisons*



naïve deconvolution



Wiener deconvolution

# Deconvolution is hard.

- Active research area.
- Even if you know the filter (non-blind deconvolution), it is still hard and requires strong *regularization* to counteract noise.
- If you don't know the filter (blind deconvolution), then it is even harder.

# In summary – *Take home messages*

- Fourier transform is awesome!
  - Make use of Fourier series for representing signal in the frequency domain.
- We can operate directly on the freq. domain faster
  - Convolution theorem.
- We can even deconvolve!
  - Well... More or less...

# Questions?

# References

## Deconvolution:

- Szeliski textbook, Sections 3.4.3, 3.4.4, 10.1.4, 10.3.
- Fergus et al., “Removing camera shake from a single image,” SIGGRAPH 2006.  
the main motion deblurring and blind deconvolution paper we covered in this lecture.
- Heide et al., “High-Quality Computational Imaging Through Simple Lenses,” TOG 2013.  
the paper on high-quality imaging using cheap lenses, which also has a great discussion of all matters relating to blurring from lens aberrations and modern deconvolution algorithms.
- Levin et al., “Understanding and evaluating blind deconvolution algorithms,” CVPR 2009 and PAMI 2011.
- Levin et al., “Efficient Marginal Likelihood Optimization in Blind Deconvolution,” CVPR 2011.  
a sequence of papers developing the state of the art in blind deconvolution of natural images, including the use Laplacian (sparsity) and hyper-Laplacian priors on gradients, analysis of different loss functions and maximum a-posteriori versus Bayesian estimates, the use of variational inference, and efficient optimization algorithms.